

Workload-Aware Data Lakehouse Maintenance Using Hierarchical Deep Reinforcement Learning

Omar Essameldin¹, Omer Karim¹, Omar Shrief¹, and Ahmed B. Zaky^{1,2}

¹Egypt-Japan University of Science and Technology (E-JUST), Computer Science and Engineering Department, Alexandria, Egypt

²Benha University, Faculty of Engineering at Shoubra, Electrical Engineering Department, Cairo, Egypt

[†] `omar.isameldin@ejust.edu.eg`, `omer.karim@ejust.edu.eg`, `omar.shrief@ejust.edu.eg`,
`ahmed.zaky@ejust.edu.eg`

[‡]Correspondence should be addressed to O.E. (`omar.isameldin@ejust.edu.eg`)

September 21, 2026

Abstract

Modern data lakehouses built on open table formats such as Apache Iceberg accumulate small files and suboptimal partition layouts as data is continuously ingested, degrading query performance over time. Existing maintenance strategies rely on hand-crafted, workload-agnostic rules that cannot adapt to shifting query distributions. We present a hierarchical multi-agent framework that jointly optimises compaction and partitioning decisions online: a rule-based meta-controller evaluates per-step urgency scores and delegates to two specialist reinforcement learning (RL) agents—a compaction agent and a partition agent—each trained with a reward signal aligned to its operational objective. Within this framework we study three neural architectures: an attention-based actor-critic (**AttentivePP0**), a feedforward actor-critic (**MLP-PP0**), and a dueling double DQN (**DDQN**), each trained under a matched configuration with five independent random seeds. Evaluation on **LAK-EGYM v5**, a simulation environment backed by a real Iceberg table on Apache Spark, comprises 216 seeded runs with paired non-parametric significance testing. The best configuration (**DDQN**) attains a mean global reward of 0.2105 (95% CI [0.2026, 0.2185], bootstrap over training seeds), exceeding the strongest of 17 heuristic baselines by +8.3% (paired permutation $p = 0.002$, Cliff's $\delta = +0.71$), with all five independent training runs individually beating that heuristic. The advantage persists on two held-out workloads exhibiting gradual drift and concurrent query mixtures. Ablation confirms a non-additive interaction between the two specialists: removing compaction collapses partition pruning from 0.225 to 0.080 because data is never rewritten into the new layout. Capacity-matched comparison shows the attention encoder confers no significant advantage ($p = 0.54$), and direct inspection of its attention weights reveals a static recency gate rather than workload-transition sensitivity. Across-seed variability differs sharply between architectures (s.d. 0.0104 to 0.0403), demonstrating that single-checkpoint evaluation is insufficient to rank architectures in this domain.

Introduction

Data lakehouses combine the scalable storage of data lakes with the transactional semantics of data warehouses through open table formats such as Apache Iceberg¹, Delta Lake², and Apache Hudi³. These formats store data as immutable Parquet files and maintain metadata-level transaction logs, enabling ACID semantics on object stores. However, continuous ingestion generates large numbers of small files, and the initial partition layout quickly becomes misaligned with the actual query distribution as workloads evolve, increasing query scan costs substantially.

State-of-practice maintenance is either manual or based on periodic compaction schedules that are decoupled from query patterns. Open-source auto-compaction heuristics¹ trigger on file-count thresholds but do not consider partition alignment. Workload-aware approaches such as learned index structures⁴ and automatic physical design tools⁵ address layout optimisation offline and are not designed for the continuous-ingestion regime of a lakehouse. Reinforcement learning has been applied to query optimisation^{6,7} and database tuning^{8,9}, but not to joint compaction and partitioning maintenance.

We argue that effective lakehouse maintenance requires *joint*, *online*, and *workload-aware* decisions over both compaction and partitioning. This paper makes the following contributions.

- **Framework.** A hierarchical framework combining a rule-based arbitrator with two learned specialists, separating the when-to-act decision from the what-to-do decision. We are explicit that the arbitration tier is *not* learned, and we quantify what that design choice costs.
- **Environment.** LAKEGYM v5, a simulation environment backed by a real Apache Iceberg table on Apache Spark with a structured, phase-shifting workload and a three-component reward function in which query latency is measured rather than modelled.
- **Architecture study.** A comparison of three neural architectures (AttentivePPO, MLP-PPO, DDQN) under a matched training configuration with five independent seeds each, evaluated with paired non-parametric tests, bootstrap confidence intervals, and effect sizes. We do not claim a universally best architecture; we report what each achieves and where they are statistically indistinguishable.
- **Mechanism analyses.** Direct measurement of the attention encoder’s behaviour, a capacity-matched control, an oracle meta-controller built on environment rollback, sensitivity analyses over every hand-chosen constant, and evaluation on two held-out workloads.
- **Negative results.** Two findings that generalise beyond lakehouse maintenance: *trajectory-identity leakage* when training offline RL on logs pooled from multiple behaviour policies — a specialisation of causal confusion in imitation learning^{10,11} to time-since-event features, together with a counterfactual probe that detects it — and a *signal-below-noise* explanation for why greedy one-step meta-control fails where cooldown-based pacing succeeds.

Related work

Learned data layout. Qd-tree¹² learns a block-partitioning scheme for analytical workloads, and SageDB¹³ and learned index structures⁴ synthesise data structures specialised to an observed data and query distribution. These systems learn a *layout* offline for a distribution assumed to be known and stationary, and reorganisation is a one-time or infrequent event. Our setting differs in two ways: the distribution shifts during operation, and every layout change is paid for continuously in rewrite cost. We therefore learn a *policy* over maintenance actions rather than a layout, and the cost of acting is part of the objective.

Reinforcement learning for database tuning. A substantial line of work applies RL to DBMS configuration: OtterTune⁸ and CDBTune⁹ for knob tuning, QTune¹⁴ for query-aware tuning, LlamaTune¹⁵ for sample-efficient search, HUNTER¹⁶ for hybrid online tuning, HWTune¹⁷ for hardware-aware tuning, CTuner¹⁸ for causal NoSQL tuning, MLETune¹⁹ for LLM-guided knob tuning, and multi-agent knob tuning for distributed databases²⁰. In all of these the action adjusts a configuration parameter; it does not rewrite data. Our actions are physically destructive rewrites of Parquet files and evolutions of the Iceberg partition specification, whose effects are delayed, partially irreversible, and interact with each other. Self-driving databases²¹ articulate the broader vision of autonomous physical design that our work instantiates for the lakehouse maintenance problem specifically.

Learned query optimisation. Neo²², SkinnerDB⁷ and Re-Join⁶ learn to choose query plans. These operate inside the execution of a single query, whereas maintenance decisions change the physical state that all subsequent queries observe. The two are complementary: a learned optimiser reduces the cost of a query given a layout, while our agents change the layout the optimiser sees.

Lakehouse storage and merge policy. Iceberg¹, Delta Lake² and Hudi³ define the transactional substrate; LST-Bench²³ provides workload-level benchmarking for log-structured tables; and the Lakehouse architecture²⁴ frames the design space. Dostoevsky²⁵ analyses space-time trade-offs in LSM-tree merging and is the closest prior treatment of compaction as an optimisation problem, but it derives a static merge policy analytically for a key-value store rather than learning an online policy for a columnar table under a shifting query distribution.

Shortcut learning in imitation and offline RL. Training a policy on logs generated by other policies is an imitation problem before it is a control problem, and it inherits that field’s characteristic failure. de Haan *et al.*¹⁰ show that a cloned policy will latch onto whatever feature best predicts the demonstrator’s action, including features that are effects of the demonstrator’s behaviour rather than causes of the correct action — a confusion that *worsens* with more expressive models, because the shortcut is easier to fit than the causal structure. The closely related “copycat” problem¹¹ arises when observation histories are available: the previous action is nearly sufficient to predict the next one, so the policy learns to repeat rather than to act. The defect we report below in our own

pipeline is an instance of this class, specialised to operational logs: when trajectories from several maintenance policies are pooled, time-since-event features identify *which* policy generated a trajectory, and the objective prefers that identity over the table state. Our contribution is the specialisation and its detection (a counterfactual probe over the suspect feature), not the phenomenon.

On the offline-RL side, conservative methods^{26,27,28} address the complementary failure — value overestimation on actions the behaviour policy never took — by penalising out-of-distribution actions, filtering them, or constraining the learned policy to the data support. We report the action coverage of our pools in Methods and explain there why we did not apply these methods.

Hierarchical and multi-agent RL. FeUdal Networks²⁹ learn a manager that sets subgoals for a worker, the option-critic architecture³⁰ learns both the options and the policy over them end to end, and the multi-agent RL literature³¹ surveys coordination mechanisms. Our meta-controller occupies the manager role but is rule-based rather than learned. We treat this as a deliberate simplification and evaluate it directly: we construct an oracle arbitrator and a sensitivity sweep over its constants to bound what the simplification costs.

Results

Corrections to the training pipeline

While constructing the multi-seed evaluation protocol reported below, we identified two defects in our offline training pipeline that materially affected previously reported results. We describe them first because they explain why the numbers in this paper differ from those in the original preprint, and because both failure modes are, in our assessment, easy to reproduce in other offline RL pipelines trained on operational logs.

Train/serve normalisation skew. Training standardised the `rows_ingested` and `ingestion_rate` features by per-dataset z -scores, whereas the deployed agent normalised them with fixed constants. Inference features were therefore 10–50 $\times$ outside the range seen during training, saturating the network into an effectively constant COMPACT_128KB policy. The apparent adaptivity of the deployed agent was an out-of-distribution artefact rather than learned behaviour.

Trajectory-identity leakage. The staleness features δ_c (steps since last compaction) and δ_p (steps since last partition change) encode *which behaviour policy generated a trajectory*: they remain near zero for `AlwaysCompact` and grow without bound for `No_Maintenance`. When advantage-weighted regression³² is trained on logs pooled across heuristics, these features provide a shortcut that identifies the generating policy, and the network clones it instead of reading table state — the mechanism known as causal confusion in imitation learning¹⁰, here driven by a feature that systems telemetry supplies as a matter of course. Holding table state fixed and varying only δ_c from 0 to 30, the predicted probability of NOOP rises from 0.00 to 0.94 — a policy that is a function of trajectory provenance rather than of the lakehouse.

Both defects are corrected by capping the staleness features, unifying normalisation between training and serving, adding counterfactual recovery trajectories, and sampling actions in a balanced manner during offline training (Methods). Critically, **the environment itself is unchanged**: re-evaluating the heuristic baselines under the corrected pipeline reproduces the published values to within 0.003 (for example `AlwaysCompact_C128`: 0.1869 $\rightarrow$ 0.1870). The changes in the RL results are therefore attributable to the pipeline corrections and not to environment drift.

Overall policy ranking

Table 1 presents mean global reward with percentile bootstrap 95% confidence intervals for all policies on the 500-step evaluation workload, together with the across-seed standard deviation. Each RL architecture was trained with five independent random seeds and each resulting agent evaluated on five episodes, giving 25 matched episodes per policy. Environment seeds are a deterministic function of (protocol, seed slot, episode) and are independent of the policy, so every policy is evaluated on identical episodes and all comparisons are paired.

DDQN attains the highest mean global reward (0.2105), ahead of the best workload-aware heuristic (`WorkloadAware Threshold`, 0.1943) by +0.0162, a relative improvement of +8.3%. The 1,000-step protocol produces a concordant ordering (Spearman $\rho = 0.964$, $p < 0.001$).

Table 1 reports two confidence intervals per policy, because the two answer different questions. The episode-level interval ([0.2050, 0.2155] for DDQN) treats the 25 episodes as the sampling unit. The seed-cluster interval ([0.2026, 0.2185]) resamples the five *training runs* with their episodes nested, which is the relevant uncertainty for a reader asking how a newly trained agent would perform, and is necessarily the wider of the two. The headline comparison

survives the stricter interval: DDQN’s seed-cluster lower bound (0.2026) still lies above the best heuristic’s seed-cluster upper bound (0.1958). We quote the seed-cluster interval in the Abstract.

We emphasise two features of this table that were absent from our original analysis. First, **not all RL configurations beat the heuristics**: MLP-PP0 ranks below three heuristic policies. The defensible claim is that the best RL configuration outperforms the best heuristic, not that reinforcement learning as a class dominates. Second, the across-seed standard deviation varies by a factor of four between architectures (0.0104 for DDQN versus 0.0403 for AttentivePP0), which is itself a result and is analysed below.

Rank	Policy	Reward	95% CI (episode)	95% CI (seed cluster)	Seed s.d.	Latency (ms)	Files	Pruning
1	DDQN (RL)	0.2105	[0.2050, 0.2155]	[0.2026, 0.2185]	0.0104	219.6	37.4	0.204
2	WorkloadAwareThreshold	0.1943	[0.1923, 0.1964]	[0.1928, 0.1958]	0.0020	323.6	49.9	0.248
3	AlwaysCompact_C128	0.1903	[0.1901, 0.1904]	[0.1901, 0.1904]	0.0002	197.9	22.1	0.000
4	AttentivePP0 (RL)	0.1823	[0.1669, 0.1963]	[0.1481, 0.2121]	0.0403	331.1	51.6	0.225
5	Threshold10_C128	0.1710	[0.1708, 0.1712]	[0.1708, 0.1712]	0.0003	203.3	26.1	0.000
6	MLP-PP0 (RL)	0.1675	[0.1579, 0.1772]	[0.1481, 0.1870]	0.0254	316.9	61.6	0.221
7	Threshold10_C64	0.1273	[0.1267, 0.1280]	[0.1264, 0.1282]	0.0011	321.3	46.8	0.000
8	CompactOnly (ablation)	0.1202	[0.1086, 0.1315]	[0.0949, 0.1455]	0.0323	314.2	48.4	0.000
9	PartitionOnly (ablation) [†]	-0.1099	[-0.1195, -0.1011]	[-0.1232, -0.1020]	0.0116	3,643	793.0	0.080

Table 1: **Policy ranking with uncertainty.** Mean global reward on the 500-step evaluation workload (eval500: 500 steps, 5 training seeds $\times$ 5 episodes = 25 matched episodes per policy). Two percentile bootstrap 95% intervals are reported (10,000 resamples each). The *episode* interval resamples the 25 episodes and answers “how precisely do we know this agent’s mean?”. The *seed-cluster* interval resamples the five training seeds with their episodes nested, and answers the question a practitioner actually asks — “how would a *newly trained* agent perform?”. It is the wider and more conservative of the two, and it is the interval quoted in the Abstract. “Seed s.d.” is the standard deviation of the five per-seed means. Hierarchical RL agents are shown in bold. Latency, file count and pruning ratio are episode-level means. [†]PartitionOnly was evaluated on 3 seeds (15 episodes); every other row uses 5 seeds.

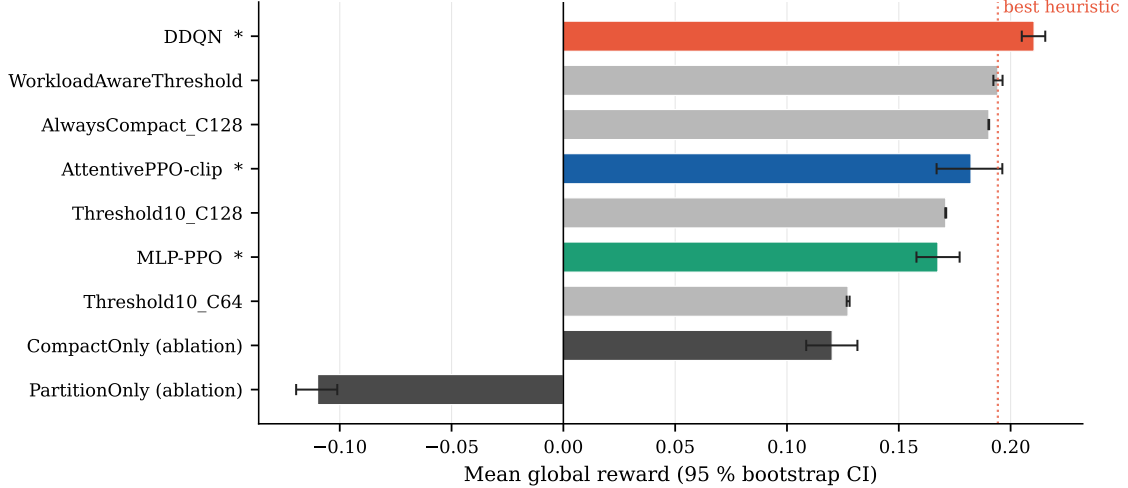


Figure 1: **Policy ranking with bootstrap 95% confidence intervals.** Bars show mean global reward on the 500-step evaluation workload; whiskers are percentile bootstrap 95% confidence intervals over 25 matched episodes. Hierarchical RL agents are coloured by architecture (coral = DDQN, blue = AttentivePP0, teal = MLP-PP0) and marked with an asterisk; heuristics are light grey and ablations dark grey. The dotted line marks the best heuristic. DDQN is the only policy whose interval lies entirely above it; MLP-PP0 falls below three heuristics.

Table 2 reports paired significance tests for DDQN against every other policy. We use both the Wilcoxon signed-rank test and an exact paired permutation test, report Cliff’s δ as a non-parametric effect size, and apply Holm–Bonferroni correction within the comparison family. DDQN significantly outperforms every other policy evaluated ($p_{\text{Holm}} \leq 0.0018$), and all five of its independently trained seeds individually exceed the best heuristic (per-seed range 0.198–0.224).

Comparison (DDQN vs.)	n	Δ	p_{Holm}	Cliff’s δ	Magnitude	Verdict
WorkloadAwareThreshold	25	+0.0162	0.0018	+0.71	large	significant
AlwaysCompact_C128	25	+0.0202	0.0018	+0.84	large	significant
AttentivePPO	25	+0.0282	0.0020	+0.44	medium	significant
Threshold10_C128	25	+0.0395	0.0018	+1.00	large	significant
Threshold10_C64	25	+0.0832	0.0018	+1.00	large	significant
<i>Comparisons that are not significant:</i>						
AttentivePPO vs. MLP-PPO	25	+0.0148	0.37	+0.33	medium	not significant
AttentivePPO vs. WorkloadAwareThreshold	25	−0.0120	0.58	+0.10	negligible	not significant

Table 2: **Paired significance tests on global reward.** Differences are paired by environment seed on the eval500 protocol; n is the number of matched episodes contributing to each comparison, resolved per comparison rather than globally. p -values are from paired permutation tests (Monte Carlo, 20,000 sign-flips above $n = 20$, so the smallest attainable raw p is 5×10^{-5}) with Holm–Bonferroni correction across the 36 comparisons in this family; Wilcoxon signed-rank tests agree on every verdict. Cliff’s δ magnitude thresholds are 0.147 (small), 0.33 (medium), 0.474 (large). MLP-PPO is significantly *worse* than the best heuristic ($\Delta = -0.0268$, $p_{\text{Holm}} = 0.0020$, $\delta = -0.71$). Note the final row, where the mean difference is negative while δ is slightly positive: **AttentivePPO** loses on the mean because two of its five seeds fail badly, but wins a bare majority of individual episode pairings. The two statistics disagree because the mean is sensitive to those two seeds and δ is not, which is itself an argument for reporting both.

Architecture comparison and training-run variability

Table 3 compares the three architectures under the matched training configuration described in Methods: identical training pools, identical epoch budgets per specialist, identical batch sizes, identical window sizes, and the same five seeds. The comparison is therefore between architecture-plus-objective pairings evaluated under one configuration; it is not a claim that any architecture is superior in general, and a different tuning budget could change the ordering (we demonstrate this directly in the attention tuning study below).

DDQN is significantly ahead of both actor-critic variants, whereas **AttentivePPO** and MLP-PPO are statistically indistinguishable from each other ($p = 0.31$). The architectures also differ on a dimension that mean performance conceals: **training-run stability**. Figure 2 shows the per-seed means directly. Three of the five **AttentivePPO** seeds exceed the best heuristic while two fail badly (0.124 and 0.157); DDQN’s five seeds are tightly clustered and all exceed it. A single-checkpoint evaluation of the kind we originally performed could therefore have reported almost any ranking of these three architectures, which is the strongest practical argument in this paper for multi-seed protocols in this domain.

Architecture	Reward	95% CI	Seed s.d.	Seed range	Params	Objective
DDQN	0.2105	[0.2050, 0.2155]	0.0104	0.198–0.224	193,739	Double DQN
AttentivePPO	0.1823	[0.1669, 0.1963]	0.0403	0.124–0.220	170,571	offline PPO-clip
MLP-PPO	0.1675	[0.1579, 0.1772]	0.0254	0.143–0.202	192,203	offline PPO-clip

Table 3: **Three-architecture comparison under matched configuration.** All three architectures were trained on identical data pools with identical epoch budgets (150 epochs compaction, 200 epochs partition), batch size 64, and the same five random seeds. “Seed s.d.” and “seed range” are computed over the five per-seed episode means and quantify training-run variability. Parameter counts are the sum over both specialists.

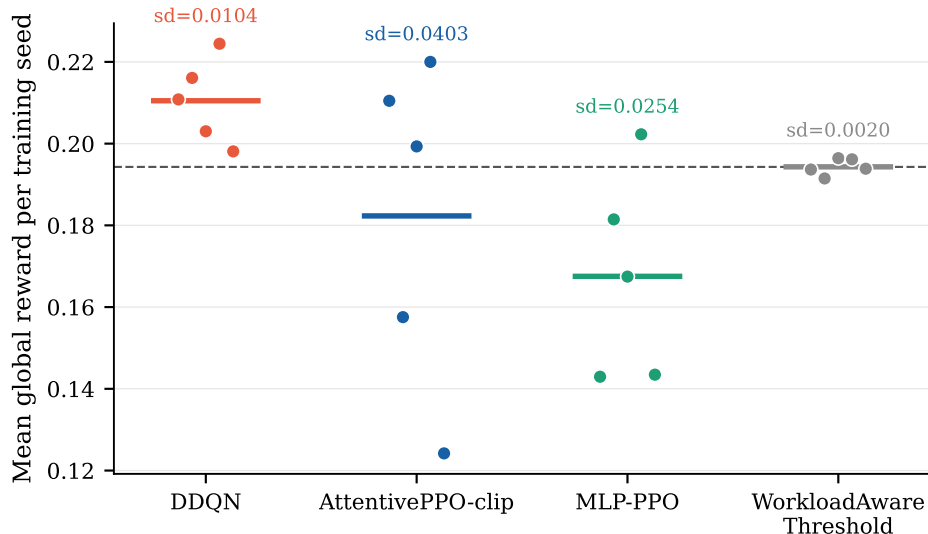


Figure 2: **Per-seed reward: the variability a single checkpoint conceals.** Each point is the mean global reward of one independently trained agent (5 episodes each); the horizontal bar is the across-seed mean and the dashed line the best heuristic. **AttentivePPO** spans 0.124–0.220, so whether it appears to beat the best heuristic depends on which seed is evaluated. **DDQN** is both higher and markedly more stable. The heuristic itself is near-deterministic (s.d. = 0.0020), reflecting environment noise alone.

Ablation: the two specialists interact non-additively

Table 4 isolates the contribution of each specialist by restricting the agent to the corresponding action sub-space. We report absolute rewards rather than percentage gains over **PartitionOnly**, because that baseline suffers uncontrolled file proliferation and a ratio against it is close to tautological.

This ablation is run on AttentivePPO, not on the headline DDQN agent. The “full hierarchical agent” row is therefore **AttentivePPO**’s 0.1823, the same value as its row in Table 1, and the ablations restrict that architecture’s specialists. We chose it when the ablation was designed and before the multi-seed results had identified **DDQN** as the stronger architecture, and we state it here rather than leaving it to be inferred from the Methods. Because the mechanism at stake — that partitioning without compaction cannot deliver pruning — is a property of the environment rather than of the encoder, we expected it to hold for **DDQN** as well, and we tested that expectation directly rather than assuming it (Table 5).

The informative comparison is against **CompactOnly**. The full agent and **CompactOnly** are physically almost identical — 51.6 versus 48.4 mean files, 331 versus 314 ms mean latency — so essentially the entire 0.0621 reward difference is attributable to partition pruning (0.225 versus exactly 0.000).

PartitionOnly establishes the interaction. Despite being the partition specialist acting alone, its pruning ratio *collapses* to 0.080, well below the 0.225 the same specialist achieves within the full agent. Without compaction, data is never rewritten into the newly declared partition layout, so declaring a partition specification does not by itself deliver pruning. Compaction is thus a prerequisite for partitioning to have any effect, and the two decisions cannot be optimised independently.

Configuration	Reward	Δ vs. full	Pruning	Files	Latency (ms)
Full hierarchical agent	0.1823	—	0.225	51.6	331
CompactOnly	0.1202	−0.0621	0.000	48.4	314
PartitionOnly	−0.1099	−0.2922	0.080	793.0	3,643

Table 4: **Specialist ablation.** Each ablation restricts the agent to one specialist’s action sub-space, holding architecture and training configuration fixed. Both differences from the full agent are significant ($p_{\text{Holm}} = 0.0022$, $|\delta| \geq 0.97$). The full agent and **CompactOnly** reach nearly identical physical table states, so their reward gap is almost entirely pruning; **PartitionOnly** demonstrates that partitioning without compaction does not deliver pruning.

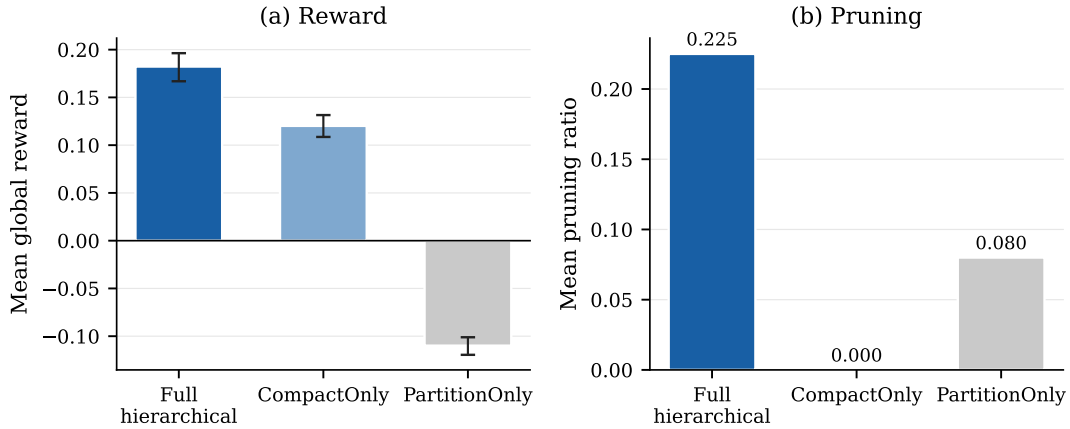


Figure 3: **Ablation of the two specialists.** (a) Mean global reward with bootstrap 95% confidence intervals. (b) Mean partition pruning ratio. **PartitionOnly** attains only 0.080 pruning despite being the partition specialist, because without compaction the data is never rewritten into the declared layout — the interaction between the two specialists is non-additive.

The same ablation on DDQN. Because the headline agent is DDQN, we repeated both ablations on it (Table 5). Every conclusion above reappears, and more sharply. Removing the partition specialist costs 0.0507 ($p_{\text{Holm}} = 0.0001$, $\delta = +1.00$, 25 paired episodes) while leaving the physical table essentially unchanged — 32.4 versus 37.4 files and 222 versus 220 ms — so once again the entire gap is pruning (0.204 versus exactly 0.000). Removing the compaction specialist collapses pruning to 0.030, further below the 0.204 the same specialist reaches inside the full agent than the 0.080 we measured for **AttentivePP0**, and the table degenerates to 1,117 files and 4.3 s queries.

We ran the DDQN **PartitionOnly** arm on three seeds with one episode each, because at 36 minutes of Spark execution per episode a five-episode design would have cost nine hours to re-establish a mechanism the environment already guarantees: pruning benefit is scaled by the fraction of data lying in the declared layout, and only compaction rewrites data into it, so an agent without a compaction action cannot reach full coverage however well it chooses partition specifications. With $n = 3$ the permutation test’s floor is $p = 0.25$, so we report this arm as a confirmation of the mechanism and an effect size, not as a hypothesis test.

Configuration	Reward	Δ vs. full	Pruning	Files	Latency (ms)
Full DDQN agent	0.2105	—	0.204	37.4	220
CompactOnly	0.1597	−0.0507	0.000	32.4	222
PartitionOnly [†]	−0.1713	−0.3766	0.030	1,117	4,278

Table 5: **Specialist ablation repeated on DDQN.** The ablation in Table 4 uses **AttentivePP0**; this one restricts the headline DDQN agent to the same action sub-spaces, evaluated on **eval500** and paired by environment seed against the full agent. **CompactOnly**: 5 seeds $\times$ 5 episodes, $p_{\text{Holm}} = 0.0001$, $\delta = +1.00$. [†]**PartitionOnly**: 3 seeds $\times$ 1 episode, paired against the corresponding episodes of the full agent; at $n = 3$ the exact permutation test cannot fall below $p = 0.25$, so this row is reported as an effect size and a mechanism check rather than a test. The conclusion is the same as for **AttentivePP0** and somewhat stronger: partitioning without compaction delivers almost no pruning (0.030 against 0.204 within the full agent).

Sensitivity to hand-chosen constants

The framework contains three sets of values chosen by hand: the meta-controller constants, the environment’s pruning matrix, and the reward weights. We vary each in turn.

Meta-controller constants. Table 6 sweeps the urgency threshold θ , the utilisation boost, and both cooldowns one at a time around the published operating point, evaluating each configuration on the same three matched episodes (one episode on each of three seeds, with the DDQN checkpoint matched to the seed). The default configuration therefore scores 0.2033 here rather than the 0.2105 of Table 1, which is the same agent and the same 500-step workload measured over 25 episodes instead of 3; the two intervals overlap substantially. Only differences *within*

this table are meaningful, because only those are paired. Reward varies by 0.0295 across the entire grid, or 14.5% of the default configuration’s value, and no configuration collapses. The published constants are reasonable but mildly conservative: a 2-step compaction cooldown would have improved reward by 8.2%. We note that the two worst configurations fall slightly below the best heuristic, so the meta-controller is robust within a sensible range rather than universally.

Parameter varied	Value	Reward	95% CI	% vs. default
(default)	$\theta = 0.35$, boost 0.15, $c_c = 4$, $c_p = 20$	0.2033	[0.1941, 0.2154]	—
Urgency threshold θ	0.25	0.1935	[0.1914, 0.1947]	−4.8
	0.30	0.1909	[0.1877, 0.1953]	−6.1
	0.40	0.2041	[0.1927, 0.2199]	+0.4
	0.45	0.2019	[0.1877, 0.2206]	−0.7
Utilisation boost	0.00	0.1967	[0.1871, 0.2108]	−3.2
	0.30	0.2071	[0.1944, 0.2212]	+1.9
Compaction cooldown c_c	2	0.2200	[0.2048, 0.2294]	+8.2
	8	0.1941	[0.1779, 0.2050]	−4.5
Partition cooldown c_p	10	0.2033	[0.1940, 0.2194]	+0.0
	40	0.1905	[0.1712, 0.2102]	−6.3

Table 6: **Meta-controller sensitivity.** One-at-a-time perturbation of every hand-chosen meta-controller constant, three matched episodes per configuration. Total spread across the grid is 0.0295 (14.5% of the default). With $n = 3$ an exact permutation test cannot reach $p < 0.25$, so this analysis is powered for *estimation* rather than hypothesis testing and we report magnitudes and intervals rather than p -values.

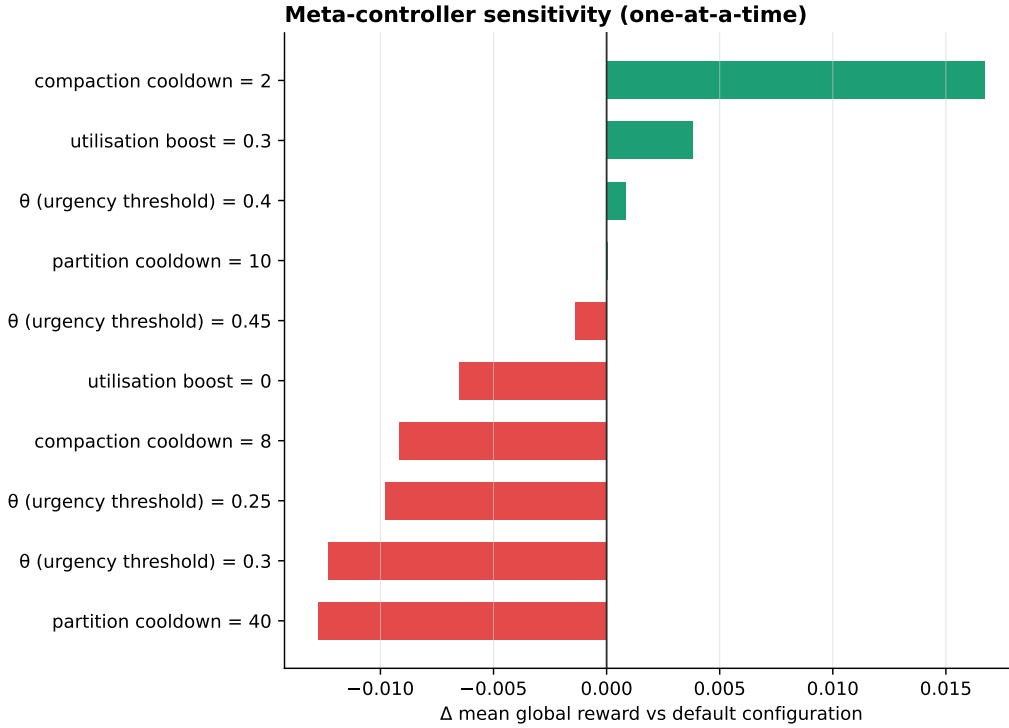


Figure 4: **Meta-controller sensitivity (tornado plot).** Deviation in mean global reward from the default configuration for each one-at-a-time perturbation, ordered by magnitude. The compaction cooldown is the most influential constant; the partition cooldown and urgency threshold follow. No perturbation causes collapse.

Pruning matrix. The environment’s query benefit is driven by a lookup table mapping (query type, partition strategy) pairs to a fraction of files skipped (Table 19). These values are assumed rather than measured — a limitation we state plainly in the Discussion. To test whether our conclusions depend on them, we scaled every

non-zero entry by $\{0.8, 0.9, 1.0, 1.1, 1.2\}$ and re-evaluated both the best RL agent and the best heuristic at each scale, so that the *gap* rather than only the absolute score can be tracked (Table 7).

This sweep uses the same reduced budget as the meta-controller sweep: three matched episodes per configuration, one on each of three seeds (Table 23). Scale 1.0 is therefore *not* the published configuration re-reported — it is the same agent and workload measured on 3 episodes rather than 25, which is why DDQN reads 0.2005 here against 0.2105 in Table 1 and `WorkloadAwareThreshold` 0.1971 against 0.1943. Each 3-episode estimate is correspondingly imprecise (DDQN at scale 1.0: 95% CI $[0.1895, 0.2196]$, which contains 0.2105). The row to read is the gap, which is paired within each scale; the absolute levels should be taken from Table 1.

The RL advantage is positive at every scale, including a 20% weaker pruning benefit. We also note a fact that bears directly on the concern that the advantage is an artefact of the pruning table: DDQN wins while achieving *lower* pruning than the best heuristic (0.204 versus 0.248, Table 1). Its advantage instead shows up in the physical state of the table — 37 files versus 50, and 220 ms versus 324 ms mean query latency on the 25-episode protocol (258 versus 309 ms on the three episodes of this sweep) — rather than in exploiting the pruning table more aggressively. We qualify the latency half of that statement below: a substantial and measured part of query latency at this scale is fixed overhead that no policy can remove, so the 104 ms difference is a difference in the scan-dependent remainder, not in total query cost.

Pruning scale	DDQN	WorkloadAwareThreshold	Gap
0.8	0.1865	0.1817	+0.0048
0.9	0.1976	0.1882	+0.0094
1.0	0.2005	0.1971	+0.0034
1.1	0.2122	0.2015	+0.0106
1.2	0.2162	0.2076	+0.0086

Table 7: **Sensitivity to the pruning matrix.** All non-zero pruning values scaled by a common factor; both the best RL agent and the best heuristic re-evaluated at each scale on matched episodes. The RL advantage remains positive throughout, so the qualitative conclusion does not depend on the exact assumed values.

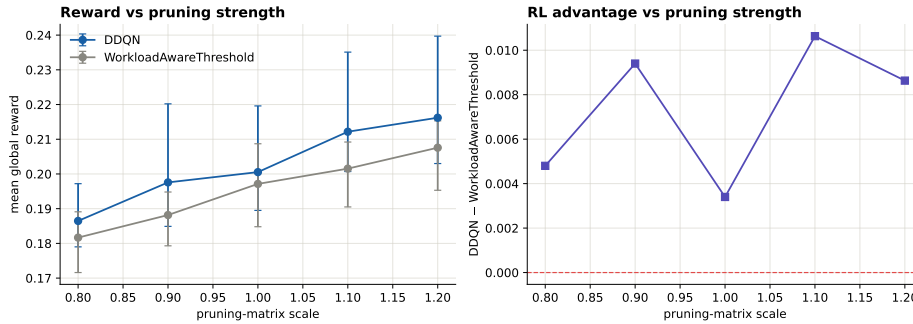


Figure 5: **RL advantage under perturbation of the pruning matrix.** (a) Mean global reward for DDQN and the best heuristic as all non-zero pruning values are scaled by a common factor; error bars are percentile bootstrap 95% intervals over the three matched episodes per scale. (b) The difference between the two curves. Both policies improve as pruning becomes more effective and the advantage of the learned policy is positive at every scale, but the size of that advantage is not monotonic in the scale factor — it dips to +0.0034 at scale 1.0 — and with three episodes per point the ordering of the gaps across scales is within noise. The claim this figure supports is the sign of the gap, not its shape.

Reward weights and cost prices. Perturbing each of the four global-reward weights by $\pm 25\%$ and renormalising leaves the policy ranking essentially unchanged (Spearman $\rho \geq 0.95$ against the published weighting in all nine variants, with DDQN ranked first under every one); the changes that occur are adjacent swaps among mid-table heuristics. Varying the three cost-model unit prices over $\{0.5\times, 1\times, 2\times\}$ in all 27 combinations never changes the cost ordering, and neither does replacing the flat per-operation compaction charge with a size-proportional one.

An oracle meta-controller does not upper-bound the rule-based design

To quantify what the rule-based arbitration tier costs, we implemented a greedy oracle meta-controller using environment snapshot and rollback. At each step the world is advanced once by ingestion, a snapshot is taken, and each of the three delegations (NOOP, compaction specialist, partition specialist) is executed as a trial branch with its *realised* reward measured by actually running the query. The Iceberg data snapshot, partition specification, simulator state and both agents’ observation windows are then restored, and the best branch is executed for real. We chose rollback over model-based lookahead because the reward depends on measured query latency, which no model in our system predicts accurately (a fitted predictor reaches only $R^2 = 0.67$), so a lookahead oracle would bound the model rather than the environment.

This experiment runs on the `std1000` protocol (1,000 steps), not the 500-step protocol used for Table 1, and the oracle’s cost — three trial branches and a rollback at every step, two hours of Spark execution per episode — limited it to a single episode on one seed. Both comparators below are therefore `std1000` numbers and are not comparable with the 0.2105 of Table 1, which is a 500-step, 25-episode mean (Table 23).

The result is a negative one, and informative. To make the comparison exact we replayed the rule-based agent on the oracle’s own environment seed, so that both controllers face an identical workload realisation rather than being compared across seed blocks. **The greedy oracle scores 0.0546 against the rule-based controller’s 0.2435 on that same realisation — it does not upper-bound the design at all.** Table 8 shows why: the per-step reward difference between the three delegations has median 0.0069, while reward noise arising from query-latency variation alone has $\sigma = 0.0120$. The selection signal lies below the environment’s own measurement noise, so the oracle largely selects whichever branch happened to draw a fast query, and the correlation between a branch’s trial reward and its realised reward is only 0.156. More fundamentally, maintenance is an investment whose payoff is delayed; a one-step realised reward cannot observe it. The two controllers agree on only 22.7% of steps.

We therefore report this as evidence that *when to act* is not a greedy optimisation problem in this domain, and that the cooldowns and urgency thresholds encode temporal structure that a myopic optimiser cannot discover. The practical bound on the rule-based design instead comes from Table 6: the best configuration in the sweep scores 0.2200 against the default’s 0.2033, so the loss attributable to suboptimal constants is approximately 8%.

Quantity	Value
Rule-based meta-controller, <i>same workload realisation</i>	0.2435
Rule-based meta-controller, mean over 5 <code>std1000</code> seeds	0.2306
Greedy one-step oracle	0.0546
Median per-step spread between the three branch rewards	0.0069
Reward noise from query-latency variation alone (σ)	0.0120
Correlation(trial reward, realised reward)	0.156
Oracle gain over always-NOOP, per step	+0.00016
Delegation agreement with the rule-based controller	22.7%

Table 8: **Why greedy meta-control fails.** All values are `std1000` (1,000 steps). The oracle selects among delegations by measured one-step reward after environment rollback. Because the spread between delegations is smaller than the environment’s latency-induced reward noise, selection is dominated by noise rather than by signal, and the delayed payoff of maintenance is invisible to a one-step objective. Two rule-based comparators are given. The first is the like-for-like comparison: the rule-based agent replayed on the oracle’s own environment seed (35101), so both controllers see the same ingestion and query stream. The second averages five `std1000` seeds drawn from a different seed block and is quoted only to show that the gap does not depend on the realisation (per-seed range 0.192–0.255). The original submission compared the oracle against the second of these, which was not a paired comparison; the first is. Neither is comparable with the 500-step numbers in Table 1.

Generalization to held-out workloads

The agents are trained on trajectories from the standard workload, in which query-profile transitions are abrupt and each phase is dominated by a single query type. We constructed two workloads that violate these assumptions and were never seen in training:

- `drift500` — the same five profiles in the same order, but transitions are linear interpolations of query-type probabilities over 50-step windows rather than step changes.

- **mixed500** — each phase samples 50/50 from two profiles simultaneously, so no single query type dominates. This directly attacks the partition specialist’s mechanism, since under a mixture there is no single correct partition specification.

Table 9 shows that the RL advantage persists. DDQN remains the top policy on both workloads, and its margin over the best heuristic is essentially unchanged (+0.0162 in distribution, +0.0140 under drift, +0.0160 under mixture). Degradation tracks pruning loss closely, and **AlwaysCompact_C128** — which never partitions and therefore has no workload-alignment mechanism to disrupt — is the only policy that does not degrade, confirming that the effect is specific to partition alignment rather than general noise. The partition-dependent policies degrade most under mixture, as predicted.

This design uses one episode per seed, against the five episodes per seed of the in-distribution protocol. With $n = 5$ paired observations an exact two-sided permutation test has a minimum achievable p of 0.0625, so this study is powered for estimation rather than hypothesis testing, and we report effect sizes and intervals rather than claiming significance.

Two distinct effect sizes are relevant here and we state both explicitly, because they carry opposite signs and have been confused in our own earlier reporting. The first is *degradation*: each policy on a held-out workload against itself in distribution, computed on per-seed means. These are negative by construction, ranging from $\delta = -0.20$ (**AttentivePPO** and **MLP-PPO** under drift) to $\delta = -0.44$ (DDQN under both shifts); the near-deterministic heuristics reach $\delta = -1.00$ because their tiny, consistent drops order perfectly across seeds, which illustrates why δ must be read next to the magnitude it describes — **AlwaysCompact_C128** loses 0.2% of its reward. The second is *superiority*: DDQN against each comparator *on* a held-out workload. These are positive: δ from +0.33 to +0.56 under drift, and from +0.44 to +0.76 under mixture against the learned and workload-aware comparators. The one exception is **AlwaysCompact_C128** under mixture, where the three paired episodes give $\delta = -0.11$ despite DDQN’s higher mean (0.1994 versus 0.1898): on that workload the two policies are not separated by this design, and we do not claim they are.

Policy	Baseline	drift500	$\Delta\%$	mixed500	$\Delta\%$
DDQN	0.2105	0.2009	−4.5	0.1994	−5.3
WorkloadAwareThreshold	0.1943	0.1869	−3.8	0.1834	−5.6
AlwaysCompact_C128	0.1903	0.1902	−0.05	0.1898	−0.2
AttentivePPO	0.1823	0.1788	−1.9	0.1638	−10.2
MLP-PPO	0.1675	0.1609	−4.0	0.1469	−12.3

Table 9: **Performance on two held-out workloads.** **drift500** replaces abrupt profile transitions with 50-step linear interpolations; **mixed500** samples two profiles concurrently in every phase. Neither was seen during training. DDQN remains the highest-scoring policy on both, and **AlwaysCompact_C128** — the only policy with no partitioning mechanism — is the only one that does not degrade.

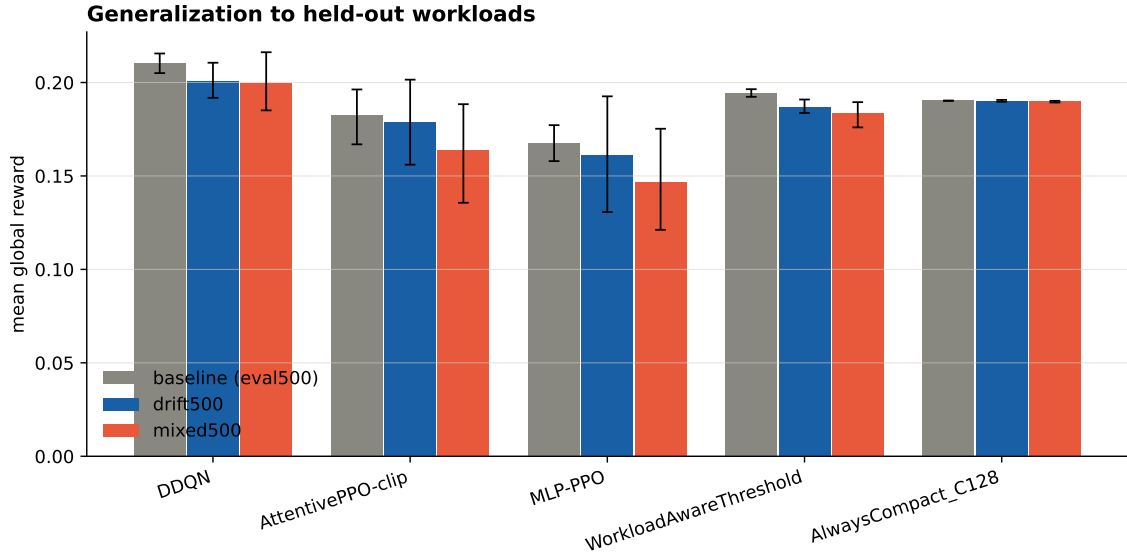


Figure 6: **Performance on held-out workloads.** Mean global reward for each policy on the in-distribution baseline (eval500, 5 episodes per seed) and on the two held-out workloads (1 episode per seed); whiskers are percentile bootstrap 95% intervals. Absolute rewards are plotted rather than relative change, so that the held-out intervals — which are wide, because each rests on one episode per seed — remain visible. DDQN is the highest-scoring policy under both shifts. Policies that rely on partition alignment degrade most under mixture; the pure-compaction heuristic AlwaysCompact_C128 is unaffected, isolating the mechanism responsible.

Online adaptation

We evaluate whether continuing to update weights during deployment helps or harms, starting each run from a frozen checkpoint and applying the appropriate online algorithm over five consecutive 500-step episodes.

Our original submission reported that online adaptation improved AttentivePPO by +20.3% while destabilising DDQN by −22.2%, and drew from that contrast a general conclusion about on-policy versus off-policy adaptation. Auditing this experiment revealed three defects, each independently sufficient to produce the reported collapse. First, the Double-DQN update regressed the model’s softmax output rather than its raw Q -values: our model returns $(\text{softmax}(Q), \max Q)$ for interface uniformity, and the online update consumed the first return value, fitting temporal-difference targets against a probability vector bounded in $[0, 1]$. Second, ε was decremented once per update call rather than once per environment step, so the “adaptive” agent explored at $\varepsilon \approx 1.0$ throughout. Third, the runs were initialised from checkpoints affected by the pipeline defects described above.

Table 10 reports the corrected experiment. All three architectures are stable. **We therefore withdraw the claim that off-policy value-based methods destabilise under online adaptation in this environment;** it was an implementation defect, not a property of the algorithm class. We also did not pursue the remedies that such a collapse would motivate (prioritised replay, shorter target-update intervals, sliding-window buffers), because there is no collapse to rescue.

Measuring adaptation requires a control run, not a comparison against Table 1. An adapting agent must explore, so the two actor-critics *sample* from their policy and DDQN acts ε -greedily, whereas the frozen evaluation behind Table 1 is deterministic ($\arg \max$) and draws on a different block of environment seeds. Comparing the two therefore confounds three things at once: learning, the action-selection rule, and the seed block. We separate them with two control arms run under the adaptation protocol itself — identical checkpoints, identical environment seeds, identical episode structure — with weight updates disabled:

- **Arm A** (all three architectures): exploration on, updates off. Against the adaptive runs it isolates *learning*.
- **Arm B** (MLP-PP0): $\arg \max$, updates off. Against arm A it isolates the *action-selection rule*; against Table 1 it isolates the *seed block*.

Arm B settles the last of these immediately: MLP-PP0 under $\arg \max$ scores 0.1680 on the adaptation seed block against 0.1675 in Table 1, a difference of 0.0005. The seed block is irrelevant, and the decomposition in Table 10 is therefore clean.

Two conclusions follow, and the second was invisible before the controls were run. First, **adaptation is neutral for all three architectures**: the learning contrasts are $+0.0012$, $+0.0058$ and -0.0053 , none significant after correction within this family of four comparisons. With five 500-step episodes each specialist receives roughly 500 transitions and 25 gradient updates, which measures fine-tuning drift rather than substantial online learning. Second, and much larger, MLP-PP0 **gains $+0.0355$ from sampling rather than taking the arg max** ($p_{\text{Holm}} = 0.0002$, $\delta = +0.72$) — six times the largest learning effect. The mechanism is the train/evaluate objective mismatch reported in the Limitations: under arg max, MLP-PP0 places 85% of its compactions on COMPACT_32KB, the target its specialist reward over-rewards and measured global reward punishes, while sampling spreads them across COMPACT_64KB and COMPACT_128KB and lifts its score to the level of the other architectures. The architecture that looked like the clearest beneficiary of online adaptation was in fact the one most damaged by deterministic action selection.

Architecture	Arm	Mean	95% CI (seed cluster)	Isolated contrast
DDQN	Table 1: arg max, no updates	0.2105	[0.2026, 0.2185]	—
	arm A: sampled, no updates	0.2080	[0.2040, 0.2140]	—
	adaptive: sampled + updates	0.2092	[0.2002, 0.2179]	learning $+0.0012$ ($p = 0.67$)
MLP-PP0	Table 1: arg max, no updates	0.1675	[0.1481, 0.1870]	—
	arm B: arg max, no updates, adaptation seeds	0.1680	[0.1479, 0.1881]	seed block $+0.0005$
	arm A: sampled, no updates	0.2035	[0.1962, 0.2104]	selection $+0.0355$ ($p = 0.0002$)
	adaptive: sampled + updates	0.2093	[0.2019, 0.2166]	learning $+0.0058$ ($p = 0.058$)
AttentivePP0	Table 1: arg max, no updates	0.1823	[0.1481, 0.2121]	—
	arm A: sampled, no updates	0.2051	[0.1927, 0.2163]	—
	adaptive: sampled + updates	0.1998	[0.1943, 0.2054]	learning -0.0053 ($p = 0.67$)

Table 10: **Online adaptation, decomposed against its own controls.** Every arm runs five consecutive 500-step episodes on each of five seeds (25 episodes) from the same checkpoints and the same environment seeds, so the contrasts are paired episode by episode. Intervals are seed-cluster bootstraps. p -values are Holm-corrected across the four contrasts in this family; the uncorrected value for the MLP-PP0 learning contrast is 0.019. The frozen arms are verified frozen: they record zero gradient updates and exactly zero loss. We report no “% of gap closed” figure: with a control arm available the comparison is a difference between measured arms, and the ratio that metric formed was unstable whenever an agent began near its reference.

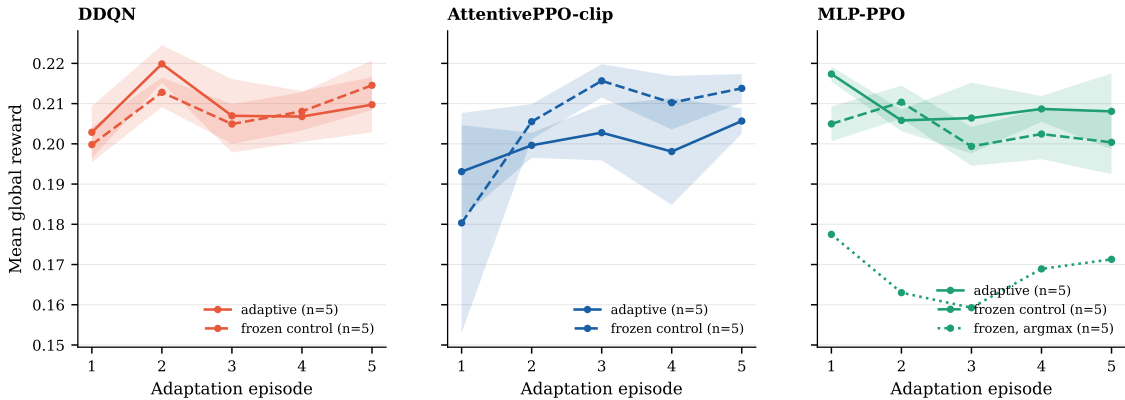


Figure 7: **Online adaptation against its controls.** Mean global reward per episode, averaged over five seeds; bands are ± 1 standard error. Solid lines are the adapting agents, dashed lines the frozen control (arm A: same checkpoints, same environment seeds, exploration on, weight updates off). The two lie within each other’s bands throughout for all three architectures, which is the direct evidence that adaptation is neutral over this window. For MLP-PP0 the dotted line adds arm B (arg max, no updates), whose separation from the others is the action-selection effect that the original frozen-versus-adaptive comparison mistook for learning.

What the attention encoder actually does

Our original submission attributed a $+3.5\%$ reward advantage to the self-attention encoder’s ability to weight recent observations differentially under workload shift. That claim was confounded and is not supported by direct

measurement.

The comparison was confounded by capacity, in opposite directions per specialist. The published MLP-PP0 had 17% *fewer* parameters than **AttentivePP0** on the compaction specialist but 42% *more* on the partition specialist. We therefore constructed a parameter-matched feedforward control, **MLP-matched**, sized to within 0.06% of **AttentivePP0** on each specialist separately (Table 11). Capacity-matched, the advantage disappears ($\Delta = 0.0049$, $p = 0.54$, $\delta = +0.16$, small).

An incidental finding is that **MLP-matched** *outperforms* the published MLP-PP0 (0.1774 versus 0.1675) while using fewer total parameters, because the published model’s capacity was mis-allocated between the two specialists. Part of the originally reported gap was therefore architecture *sizing* rather than architecture family.

Model	Reward	95% CI	Params
AttentivePP0	0.1823	[0.1669, 0.1963]	170,571
MLP-matched	0.1774	[0.1664, 0.1885]	170,667
MLP-PP0 (published sizing)	0.1675	[0.1579, 0.1772]	192,203

Table 11: **Capacity-matched attention control.** MLP-matched is a feedforward network sized to within 0.06% of **AttentivePP0** on each specialist. The difference between **AttentivePP0** and **MLP-matched** is not significant ($p = 0.54$, Cliff’s $\delta = +0.16$), so the previously reported attention advantage does not survive capacity matching.

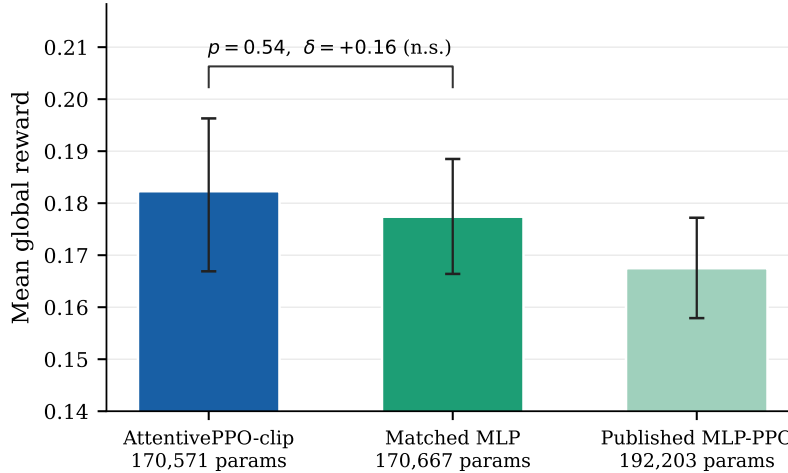


Figure 8: **Attention versus a parameter-matched feedforward control.** Mean global reward with bootstrap 95% confidence intervals. Matching capacity to within 0.06% eliminates the previously reported attention advantage. The published MLP-PP0 underperforms both, reflecting mis-allocated capacity between the two specialists rather than a property of feedforward encoders.

Direct measurement of the attention weights. We captured multi-head attention scores at every step of frozen 1,000-step rollouts across three seeds (Table 12, Fig. 9a). The partition specialist places 51.7% of its attention mass on the single most recent timestep of a 20-step window (uniform would be 5.0%), while the compaction specialist is only 1.29 $\times$ uniform — close enough to flat that the global average pooling which follows performs a similar function.

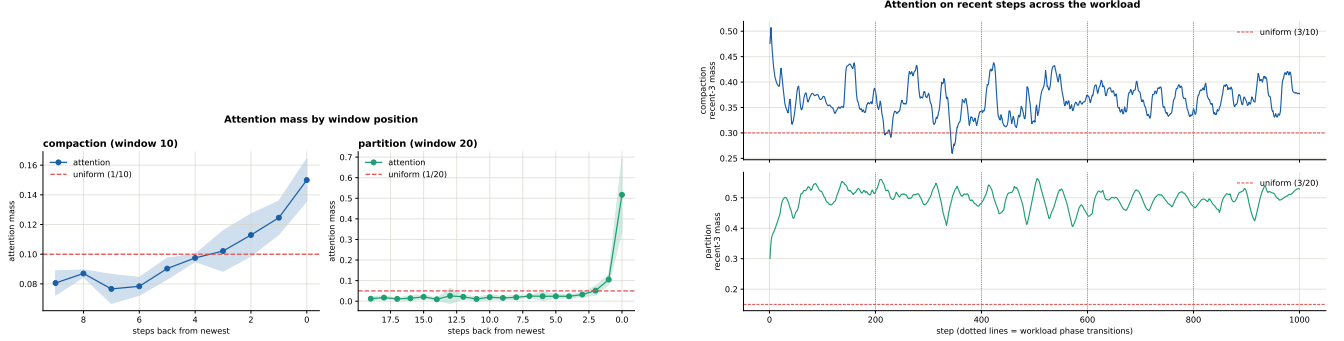
Attention concentrated on one position is a *recency gate*, not sequence modelling. Furthermore, if the encoder were exploiting workload structure, its focus should shift at profile transitions. It does not: around the four transitions the change in recency mass is negligible and inconsistent in direction across seeds ($\Delta = -0.0066$ for compaction, $+0.0031$ for partition; Fig. 9b).

Three further observations are coherent with this interpretation and are reported in full in the supplementary sweep. A feedforward network can express a fixed positional weighting, which explains the capacity-matched tie. Increasing the encoder to 7 $\times$ the parameters changes nothing (0.1830), so the mechanism is not capacity-limited. Doubling the context window *hurts* (0.1765), because extending a static profile only adds diluting positions.

Exploratory tuning. We additionally asked whether attention improves under a configuration better suited to it. Of the variants tested, only a longer training budget helped, and only modestly: $+0.0058$ ($p = 0.11$, not

Specialist	Uniform	Newest	-1	-2	-3
Compaction ($W = 10$)	0.100	0.150	0.125	0.113	0.102
Partition ($W = 20$)	0.050	0.517	0.105	0.051	0.032

Table 12: **Attention mass by position.** Mean attention mass assigned to the most recent timesteps of the observation window, averaged over three seeds and 1,000 steps each. The partition specialist concentrates over half its attention on a single position; the compaction specialist is only $1.29\times$ uniform. Neither profile changes meaningfully at workload transitions.



(a) **Attention mass by window position.** Both specialists exhibit a monotone recency profile with no structure beyond the most recent steps.

(b) **Recency mass at workload transitions.** Change in recency focus around the four profile transitions is negligible and inconsistent in sign across seeds.

Figure 9: **The attention encoder behaves as a static recency gate.** Attention scores captured at every step of frozen 1,000-step rollouts across three seeds.

significant), reaching parity with the best heuristic (0.1949 versus 0.1943) but remaining below DDQN. We state explicitly that DDQN was *not* given an equivalent tuning sweep, so this result means that attention can be brought closer with tuning, not that the architectures are equivalent under equal tuning effort. This study is exploratory and is deliberately separated from the matched-configuration comparison in Table 3.

Computational cost

Table 13 reports per-step inference latency measured over 10,000 randomised observations on CPU only. The rule-based meta-controller costs approximately $2\mu\text{s}$ and is effectively free. Notably, **AttentivePP0** is $3\times$ slower than MLP-PP0 despite having *fewer* parameters, because self-attention is quadratic in window length — a deployment consideration that parameter counts do not reveal.

Component	Median	p95	Params
Meta-controller (rule-based urgency)	0.0019 ms	0.003 ms	—
MLP-PP0 — full hierarchical step	3.48 ms	4.06 ms	192,203
DDQN — full hierarchical step	4.80 ms	6.28 ms	193,739
AttentivePP0 — full hierarchical step	10.73 ms	12.57 ms	170,571

Table 13: **Per-step inference latency.** Measured over 10,000 randomised observations on an Intel i7-1165G7 with no GPU present. A full hierarchical step comprises the meta-controller decision plus one specialist forward pass. At $\sim 5\text{ ms}$ against a mean query latency of $\sim 344\text{ ms}$, policy inference costs approximately 1.4% of a single query.

Training is inexpensive in absolute terms (Table 14). We report hardware per row because wall-clock is not comparable across hardware; the deployed **AttentivePP0** variant was trained on CPU.

Table 15 regenerates the cost model with the RL infrastructure term included, so that the comparison with heuristics is like-for-like. Inference is charged at $\$0.096/\text{h}$ CPU and offline training at $\$0.526/\text{h}$ GPU amortised over 100 deployment episodes.

Variant	Hardware	Params	Per agent	All 5 seeds
DDQN	Tesla T4	193,739	2.26 min	11.3 min
MLP-PP0	Tesla T4	192,203	1.73 min	8.7 min
AttentivePP0	local CPU	170,571	3.30 min	16.5 min

Table 14: **Offline training cost per deployable agent.** A deployable agent comprises both specialists. Wall-clock is the mean over five seeds. Hardware differs by variant and wall-clock is therefore *not* comparable across rows; we report it per row rather than normalising.

The saving relative to no maintenance survives: -82.4% for DDQN, against -82.9% claimed originally, and the RL infrastructure term is 0.003% of the total. **However, RL is not the cheapest policy.** `Threshold10_C128` costs less (\$8.37 versus \$8.79) because it performs fewer compactions (124 versus 173 per episode). Under this cost model no RL agent repays its training cost relative to that heuristic. Reinforcement learning buys reward — better pruning, lower latency, and robustness to workload shift — not minimum dollar cost, and we state this plainly rather than presenting cost as a further advantage.

Policy	Query	Compact	Storage	Inference	Training	Total	vs. no-maint.
<code>Threshold10_C128</code>	3.31	0.62	4.45	—	—	8.37	-83.2%
DDQN	3.44	0.87	4.49	0.0001	0.0002	8.79	-82.4%
AttentivePP0	4.98	0.82	4.57	0.0003	0.0003	10.37	-79.2%
<code>Threshold10_C64</code>	5.71	0.62	4.57	—	—	10.90	-78.1%
<code>WorkloadAwareThreshold</code>	5.73	1.05	4.57	—	—	11.35	-77.2%
MLP-PP0	6.37	0.58	4.67	0.0001	0.0002	11.62	-76.7%
<code>AlwaysCompact_C128</code>	3.48	5.00	4.43	—	—	12.92	-74.1%
<code>No_Maintenance</code>	42.98	0.00	6.88	—	—	49.86	0%

Table 15: **Estimated cloud cost per 1,000-step episode (USD).** Operational terms use the cost model in Methods; the inference and training columns are new and make the comparison with heuristics like-for-like. The RL infrastructure term is 0.003% of the total for DDQN. Note that the cheapest policy is a heuristic, not an RL agent.

How much of measured query latency is scan work?

Query latency enters the global reward directly, and at this simulation’s scale a reader is entitled to ask how much of it is actual scan cost and how much is the fixed price of submitting a Spark job. We measured this directly. Holding the data exactly constant — 20,000 rows, 1.7 MB consolidated — we rewrote the table at file counts from 14 (the fewest the `COMPACT_128KB` target can produce for that volume) to 448, and timed all five query templates 30 times at each, with the same cache-clearing discipline the environment uses.

Two floors bound the fixed cost. A `SELECT 1`, which touches no table, takes 55 ms; a query against a single-row table — catalog resolution, planning, job submission, one file opened, no scan work — takes 93 ms averaged over the five templates. **Between a quarter and a half of the latencies reported in Table 1 is therefore fixed overhead that no maintenance policy can remove:** 42% for DDQN at 37 files, 47% for `AlwaysCompact_C128` at 22 files, and 28–29% for the policies that leave 50–62 files behind. The reviewer-anticipated concern is real, and we now state it as a limitation.

It does not, however, make the latency term constant. Above the floor, latency is almost perfectly linear in file count over the measured range (Table 16; ordinary least squares gives $135 + 4.99 \times \text{files ms}$ with $r = 0.999$), rising from 242 ms at 21 files to 2,380 ms at 448. The consequence is different from, and more interesting than, a constant latency term: **at this scale the latency term is very nearly a second measurement of the file-count term.** The two components of the global reward that we weight independently are in fact strongly collinear here, so the effective objective is weighted towards file count more heavily than the nominal weights suggest — which is a defect of the reward decomposition at micro-scale, and one that would relax at production volumes where per-file overhead is amortised over far more data per file. We report it as such rather than as a property of the method.

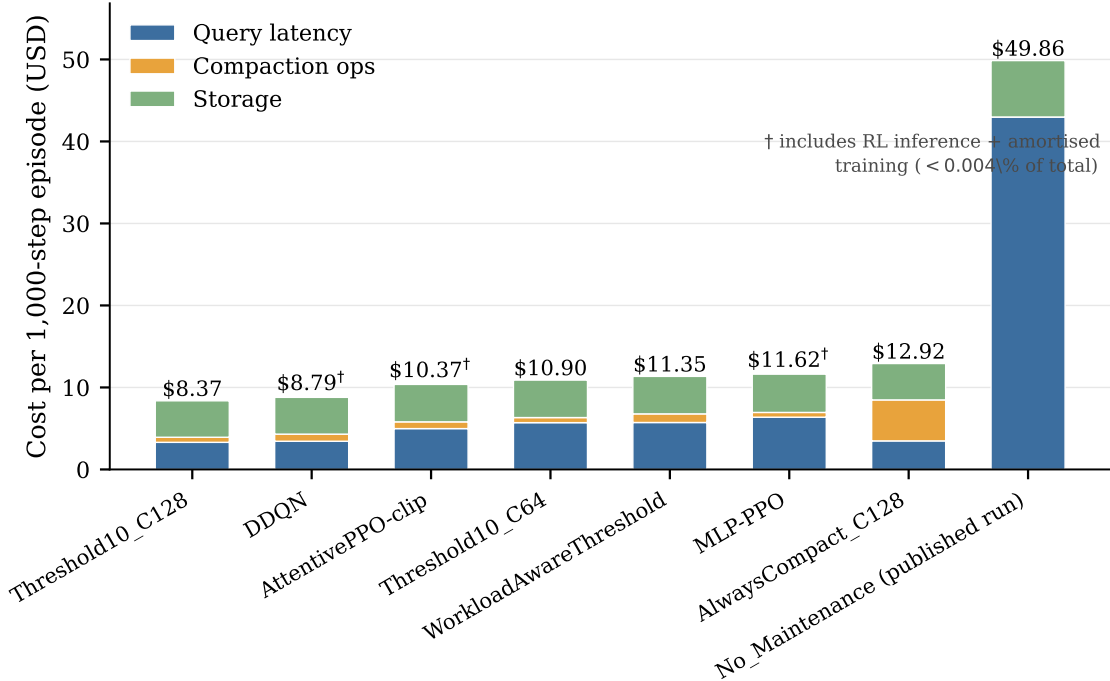


Figure 10: **Cost decomposition per 1,000-step episode.** Stacked bars show query, compaction and storage cost. `No_Maintenance` is dominated by query cost from scanning thousands of small files; `AlwaysCompact_C128` is dominated by compaction cost. Policies marked $\dagger$ additionally carry RL inference and amortised training cost, which is under 0.004% of the total and not visible at this scale.

Files	14	21	28	56	112	448
Mean latency (ms)	265	242	243	394	689	2,380

Table 16: **Query latency against file count at constant data.** The same 20,000 rows (1.7MB) rewritten into a varying number of files; mean of the median latency of each of the five query templates, 30 timed repetitions each. Reference floors: `SELECT 1` 55 ms, single-row table 93 ms. A fit over the full sweep (which also includes 224 files) gives $135 + 4.99 \times \text{files}$ ms, $r = 0.999$. The slope is specific to this data volume — bytes per file differ from the policies’ operating points — so we use the sweep for the shape of the dependence and the floors for the fixed cost, and do not extrapolate the slope to Table 1.

Discussion

Joint control is necessary, and the interaction is non-additive. Partitioning without compaction does not deliver pruning: the partition specialist acting alone reaches only 0.080 pruning against the 0.225 the same specialist achieves within the full agent, because data is never rewritten into the declared layout. We observe this on both architectures we ablated — `AttentivePPO` (Table 4) and `DDQN`, where the collapse is sharper still at 0.030 against 0.204 (Table 5) — which is what we would expect of a mechanism that belongs to the storage layer rather than to the policy. This is the clearest architectural argument in the paper, and it has a direct implication for system design: an auto-optimisation subsystem that triggers compaction and clustering independently, on separate thresholds, cannot exploit this interaction.

Which architecture wins is determined by the training objective, not the encoder. Under a matched configuration, `DDQN` significantly outperforms both actor-critic variants, while `AttentivePPO` and `MLP-PPO` — which share an objective and differ only in encoder — are statistically indistinguishable. Capacity-matched, the attention encoder confers no significant advantage, and direct inspection shows it behaves as a static recency gate that does not react to the workload transitions it was supposed to detect. We therefore do not present these three architectures as a ranking to be generalised. They are three instantiations evaluated under one configuration, and we report where they differ, where they do not, and what mechanism explains it.

Training-run variability is a first-class result. The across-seed standard deviation differs fourfold between architectures, and for **AttentivePPO** the range spans whether or not the agent beats the best heuristic. In a domain where a single evaluation episode costs several minutes of real Spark execution, the temptation to evaluate one checkpoint is strong; our own original submission did exactly that. Figure 2 is the direct evidence that this is not sufficient.

How a policy is queried can matter more than whether it keeps learning. Our original submission compared adapting agents against frozen ones without noticing that the two were queried differently — the adapting agent samples from its policy because it must explore, the frozen one takes the arg max. Controlling for that (Table 10) turns the apparent benefit of online adaptation for MLP-PPO into a benefit of stochastic action selection, six times larger than anything learning contributed. The reason is specific and, we think, general: when a specialist’s training reward is misaligned with the objective it is evaluated on, the arg max commits fully to the misalignment — here, 85% of compactions on **COMPACT_32KB** — while sampling hedges across actions and recovers much of the loss. A practitioner reading only mean reward would conclude that this agent needs online learning. It does not; it needs either an aligned reward or a stochastic execution policy. We report this as the clearest methodological lesson of the revision: an evaluation protocol must hold action selection fixed across the conditions it compares, or the comparison measures the protocol.

Greedy meta-control fails for a measurable reason. The oracle experiment (Table 8) is a negative result we consider transferable. When the per-step difference between candidate actions is smaller than the environment’s own measurement noise, a greedy selector optimises noise. In such settings the value of a rule-based arbitrator is not that it approximates the greedy optimum but that its cooldowns encode temporal structure — the delayed payoff of maintenance — which a one-step objective cannot represent.

Trajectory-identity leakage in offline RL on operational logs. The leakage described in the Corrections subsection is a specific instance of a documented phenomenon — causal confusion in imitation learning¹⁰, and its “copycat” variant when observation histories are available¹¹ — and we present it as such rather than as a new failure mode. What we believe is new, and what makes it worth reporting to a systems audience, is the specialisation: in operational logs, the shortcut is not the logged previous action but *time since an event*. Any feature counting steps since a maintenance operation encodes the identity of the behaviour policy that generated the trajectory whenever logs are pooled across policies with different action frequencies, and such features are standard in systems telemetry precisely because they are cheap and informative. Advantage-weighted regression and related behaviour-cloning objectives exploit that shortcut in preference to the state features one intends them to learn. The practical consequence is a detection recipe: aggregate validation metrics cannot see it — the cloned policy predicts held-out actions well, because it has learned the behaviour policy that produced them — and a counterfactual probe is required, holding the table state fixed and sweeping only the suspect feature. We report this as the finding most likely to transfer outside lakehouse maintenance, since pooling logs from several operational policies is the normal way such datasets are assembled.

Relationship to commercial auto-optimisation. A direct empirical comparison with Databricks, Snowflake or BigQuery auto-optimisation is not possible: the optimisers are proprietary and inseparable from their execution engines, and cannot be reproduced against our environment. We restrict ourselves to a qualitative architectural comparison based on publicly documented behaviour, and we make no performance claim. Commercial auto-optimisation is predominantly threshold- and statistics-driven, triggering on file-size and file-count targets and, for clustering, on measured deviation from a target ordering. Our contribution is not a better threshold but the framing of maintenance as a sequential decision problem under a single objective. Two structural differences follow. First, file management and clustering are typically maintained as separate subsystems with independent triggers, which the ablation above suggests forgoes a non-additive interaction. Second, these systems reorganise data within a fixed physical scheme, whereas our action space includes evolving the partition specification itself. Conversely, commercial systems optimise a robust proxy (file size, clustering depth), whereas we optimise a composite reward whose weights we must justify — a strength in expressiveness and a weakness in that the weights are a design choice. An empirical comparison would require running an identical workload on each platform and remains valuable future work.

When is RL worth it? We think the honest recommendation is conditional. Under our own cost model a well-chosen heuristic is cheaper (\$8.37 versus \$8.79) and far simpler, and for a static workload **AlwaysCompact_C128** or **Threshold10_C128** is a strong choice; we say so explicitly. The learned policy earns its complexity when the workload *shifts*, which is where partition alignment matters and where Table 9 shows the advantage persisting. The overhead of the learned policy itself is small: agents act on fewer than 24% of steps, and the cost of deciding not to act is ~5 ms against ~344 ms of query latency. Idleness is the correct behaviour rather than wasted computation.

Extensibility of the action space. Finer-grained maintenance actions — Z-ordering, per-partition file-size

targets, partial compaction — raise a combinatorial concern. We argue the hierarchy is the mechanism that contains it: because specialists own disjoint sub-spaces, adding a Z-ordering specialist adds one delegation target and one action set, so growth is additive in the number of specialists rather than multiplicative. Within a specialist, finer granularity is better served by continuous control than by enumeration; our discrete $\{32, 64, 128\}$ KB targets are a coarse discretisation of a continuous quantity, and target choice demonstrably matters. Partial compaction is the genuinely hard case, since selecting a subset of files is combinatorial in the number of files; we suggest it is better expressed as a parameterised action (a predicate over file size and age) than as a set-valued action. These are design arguments, not results.

Limitations. We state the following plainly. (i) *Scale, and what it does to the reward.* LAKEGYM v5 is backed by a real Iceberg table on real Spark with genuine Parquet files, genuine compaction, genuine partition evolution and *measured* query latency, but the tables are small: over a 1,000-step episode the table averages 4.4 MB and peaks at 8.3 MB under a maintaining policy (13.8 MB when maintenance is withheld). We measured what this implies for the reward rather than leaving it as an assertion (see *How much of measured query latency is scan work?*). Two consequences follow, and both qualify our results. First, 28–47% of the query latency in Table 1 is fixed overhead — planning, job submission, catalog resolution — that no policy can remove, so the latency differences we report are differences in a minority share of the total. Second, above that floor latency is almost perfectly linear in file count at this scale ($r = 0.999$), which means the latency and file-count terms of the global reward are strongly collinear and the effective objective is weighted towards file count more than the nominal weights imply. Neither effect changes the ranking — DDQN wins on both terms — but both would behave differently at production volumes, where a file holds orders of magnitude more data and scan cost dominates per-file overhead. External validity to production-scale deployments remains untested. (ii) *The pruning matrix is assumed, not measured.* We tested sensitivity to it (Table 7) and the conclusions hold, but we did not instrument Spark’s scan metrics to measure files-skipped per query. Doing so is the correct resolution and is required future work. (iii) *Train/evaluate objective mismatch.* Specialists are trained on `compact_reward` and `partition_reward` but evaluated on `global_reward`, and these are not aligned in one specific respect: the compaction reward’s target-size bonus favours `COMPACT_32KB` at low ingestion, whereas measured global reward strongly favours larger targets. This is not hypothetical — our MLP-PP0 agent selected `COMPACT_32KB` for 99 of its 103 compactions, a direct consequence of the mismatch and a probable contributor to its underperformance. We report this as a design flaw in the reward decomposition. (iv) *Statistical power.* The sensitivity sweep ($n = 3$ per configuration) and the generalization study ($n = 5$) are powered for estimation rather than hypothesis testing, with permutation-test floors of $p = 0.25$ and $p = 0.0625$ respectively. We report intervals and effect sizes for these and reserve significance claims for the main comparison. (v) *Unequal tuning effort.* The attention tuning study has no counterpart for DDQN, so it establishes that attention can be brought closer with tuning, not that the architectures are equivalent under equal effort. (vi) *Schema.* All experiments use a single table schema. The compaction signal is schema-independent and should transfer; the partition signal depends on the cardinality of the partitioning column, so a substantially different schema would change achievable pruning. This is argued, not demonstrated. (vii) *Short adaptation window.* The online experiments span five episodes and roughly 25 gradient updates per specialist, which measures fine-tuning drift rather than substantial online learning. The neutral result in Table 10 is therefore a statement about this window and not about whether these architectures can learn online at all; establishing the latter needs a budget an order of magnitude larger. (viii) *No conservative offline objective.* The specialists are trained with standard Double DQN and offline PPO-clip. We report action coverage (Table 20) to show that the out-of-distribution regime these methods guard against is largely absent from our pools, but coverage is not density: the partition specialist learns five actions from 4,990 transitions logged by three behaviour policies, with 336–371 examples of each non-NOOP action. A conservative objective (CQL²⁶, IQL²⁷, BCQ²⁸) would be the safer choice at that sample size, and we have not measured what it would change.

Methods

The LakeGym v5 environment

LAKEGYM v5 simulates a production data lakehouse backed by a real Apache Iceberg table managed via PySpark, with an Iceberg REST catalog and S3-compatible object storage. Each discrete time step executes: (i) ingest a batch of rows into the Iceberg table, (ii) the active policy selects one of eight discrete actions, (iii) a query is drawn from the current workload profile and executed against the table, (iv) three reward signals are computed from the resulting table state and the *measured* query latency, and (v) an observation vector is constructed.

We emphasise the boundary between what is real and what is simulated. Real: the Iceberg table, the Parquet files, the `rewrite_data_files` compaction, partition-spec evolution, and query latency, which is measured by

executing the query. Simulated: the data, which is synthetic; the query set, which comprises five templates; the scale, which averages 4.4MB and peaks at 8.3MB over an episode rather than reaching production volumes; and the pruning benefit, which is applied from the assumed matrix in Table 19 rather than measured from scan metrics.

Table schema. The table contains eight columns: `event_ts` (timestamp), `sensor_id` (string), `region` (string), `event_type` (string), `temperature` (float), `humidity` (float), `status` (string), and `payload` (string, length 100). Each row occupies approximately 200 bytes; the default target file size is 64KB, corresponding to roughly 50 rows per Parquet file.

Action space. The joint action space consists of eight discrete actions (Table 17). Compaction actions rewrite small files into consolidated Parquet files of the specified target size; partition actions change the Iceberg partition specification; NOOP takes no maintenance action.

Action	Type	Cost
NOOP	—	0.0
COMPACT_32KB	Compaction	0.9
COMPACT_64KB	Compaction	1.0
COMPACT_128KB	Compaction	1.1
PARTITION_HOUR	Partition	2.0
PARTITION_REGION	Partition	2.0
PARTITION_EVENT_TYPE	Partition	2.0
REMOVE_PARTITION	Partition	0.7

Table 17: **Action space.** All eight discrete actions available to the joint agent with their associated normalised cost used in the reward function.

Observation space. The observation at step t includes: file count f_t , block utilisation u_t , rows ingested r_t , partition pruning ratio p_t , running average pruning ratio $\bar{p}_t$, partition skew s_t , query latency ℓ_t , query-type history fractions (five values), steps since last compaction δ_c , and steps since last partition change δ_p . Following the correction described in Results, both staleness features are capped and all features are normalised with constants shared between training and serving.

Workload plans. The v5 standard workload runs for 1,000 steps divided into five equal query-profile phases of 199 steps each (`time_heavy` $\rightarrow$ `region_heavy` $\rightarrow$ `mixed` $\rightarrow$ `type_heavy` $\rightarrow$ `scan_heavy`). Each profile specifies query-type probabilities over five templates (Table 18). Ingestion rate follows six phases: warm-up (40 rows/step, steps 1–100), normal load (75, steps 101–300), burst (225, steps 301–500), recovery (55, steps 501–700), normal load 2 (75, steps 701–900), and wind-down (35, steps 901–1,000). The v5 evaluation workload is a 2 $\times$ -compressed 500-step version. Figure 11 illustrates the schedule.

The two held-out workloads used in *Generalization to held-out workloads* extend this specification. `drift500` replaces abrupt profile transitions with linear interpolation of query-type probabilities over 50-step windows; `mixed500` samples each step from a 50/50 mixture of two profiles. Both are expressed as additional fields in the workload plan and are backward compatible with the standard plan.

Profile	Time range	Region filter	Sensor lookup	Type filter	Full scan
<code>time_heavy</code>	0.60	0.10	0.10	0.10	0.10
<code>region_heavy</code>	0.10	0.60	0.10	0.10	0.10
<code>mixed</code>	0.20	0.20	0.20	0.20	0.20
<code>type_heavy</code>	0.10	0.10	0.10	0.60	0.10
<code>scan_heavy</code>	0.10	0.10	0.10	0.10	0.60

Table 18: **Query workload profiles.** Each profile specifies the sampling probability for each of five query templates. The dominant query type is shown in bold for focused profiles.

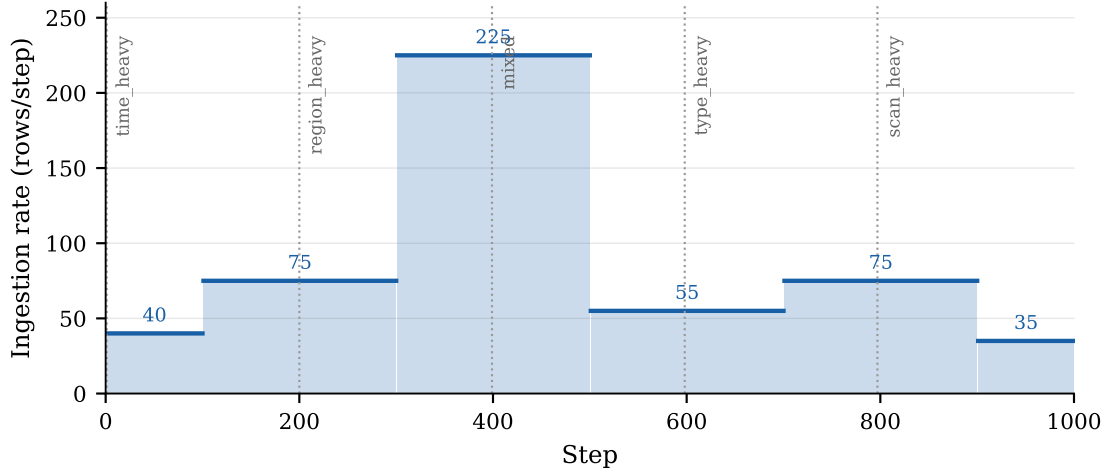


Figure 11: **Workload schedule for the 1,000-step standard plan.** Step function shows the ingestion rate (rows/step) across six phases, peaking at 225 rows/step during the burst window. Dotted vertical lines mark the five query-profile boundaries, which are independent of the ingestion phases so that storage pressure and query-mix shift are not confounded.

Pruning matrix. File pruning at query time is applied from a lookup table indexed by (query type, partition strategy) (Table 19). `TIME_RANGE` queries prune 92% of files when `HOURLY`-partitioned; `FULL_SCAN` is irreducible under all strategies. **These values are design assumptions, chosen to reflect the semantic alignment between predicate and partition key. They were not measured from Spark scan metrics.** We report a sensitivity analysis over them in Table 7 and identify their empirical measurement as required future work.

Query type	Hour	Region	Event type	None
Time range	0.92	—	—	—
Sensor lookup	0.75	—	—	—
Region filter	—	0.67	—	—
Type filter	—	—	0.67	—
Full scan	—	—	—	—

Table 19: **Partition pruning matrix (assumed).** Fraction of Parquet files skipped at query time for each query-partition combination. Dashes indicate zero pruning. Only semantically aligned pairs achieve non-zero pruning. These values are assumptions of the environment design rather than measurements; see Limitations.

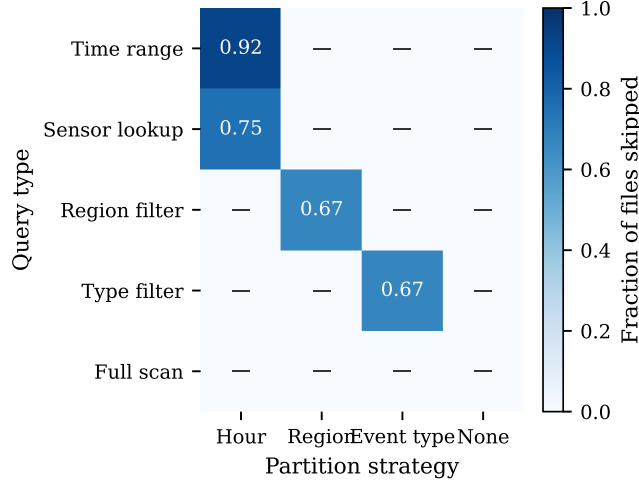


Figure 12: **Partition pruning matrix.** Fraction of Parquet files skipped at query time for each query-type-partition-strategy combination. Cell colour encodes the pruning ratio; dashes indicate zero pruning. Time-range queries achieve the highest pruning (92%) under hourly partitioning; full-scan queries are irreducible under all strategies. Values are assumed rather than measured.

Reward function

The reward function comprises three independently computed signals.

Compact reward (Eq. 1):

$$r_c = -w_f \tilde{f} + w_u u - w_c \tilde{a}_c + w_\tau \tau(a, r), \quad (1)$$

where $\tilde{f} = \min(f/F_{\max}, 1)$, u is block utilisation, $\tilde{a}_c$ is the normalised compaction action cost, and $\tau(a, r)$ is the target-size bonus depending on action a and ingestion rate r . Weights: $w_f = 0.25$, $w_u = 0.20$, $w_c = 0.10$, $w_\tau = 0.45$.

Partition reward (Eq. 2):

$$r_p = w_p p + w_{\Delta p} \Delta p - w_s \tilde{s} - w_{cp} \tilde{a}_p, \quad (2)$$

where p is the current partition pruning ratio, $\Delta p = \bar{p}_t - \bar{p}_{t-1}$ is the pruning improvement, $\tilde{s}$ is normalised partition skew, and $\tilde{a}_p$ is the normalised partition action cost. Weights: $w_p = 0.55$, $w_{\Delta p} = 0.25$, $w_s = 0.10$, $w_{cp} = 0.10$.

Global reward (Eq. 3):

$$r_g = -w_\ell \tilde{\ell} - w_f \tilde{f} + w_p p + w_u u, \quad (3)$$

where $\tilde{\ell} = \min(\ell/L_{\max}, 1)$ with $L_{\max} = 15,000$ ms. Weights: $w_\ell = 0.30$, $w_f = 0.20$, $w_p = 0.30$, $w_u = 0.20$. The global reward is the primary evaluation target; compact and partition rewards serve as dedicated training signals for the respective specialists.

These weights were chosen to balance the four components on a common scale rather than derived from a cost measurement. We therefore report a sensitivity analysis in which each weight is perturbed by $\pm 25\%$ and all four renormalised, with the global reward recomputed per step from logged metrics and all policies re-ranked. We also note explicitly in Limitations that training on r_c and r_p while evaluating on r_g constitutes an objective mismatch, and identify the specific term responsible.

Hierarchical architecture

The framework comprises three tiers: the LAKEGYM v5 environment, a rule-based meta-controller, and two specialist RL agents coordinated through a shared model registry (Fig. 13a). The arbitration tier is *not* learned; the two specialists are.

Meta-controller. The meta-controller computes two urgency scores at each step and delegates:

$$\delta = \begin{cases} \text{COMPACTION} & \text{if } u_c \geq u_p \geq \theta \\ \text{PARTITION} & \text{if } u_p > u_c \geq \theta \\ \text{NOOP} & \text{otherwise,} \end{cases} \quad (4)$$

where $\theta = 0.35$ is the minimum urgency threshold.

Compaction urgency u_c is a piecewise-linear function of file count f :

$$u_c(f) = \begin{cases} 0.95 & f \geq 96 \\ 0.60 + 0.35 \cdot \frac{f - 36}{60} & 36 \leq f < 96 \\ 0.10 + 0.50 \cdot \frac{f - 12}{24} & 12 \leq f < 36 \\ 0 & f < 12, \end{cases} \quad (5)$$

with a +0.15 boost when block utilisation falls below 0.55. A 4-step cooldown prevents repeated compaction.

Partition urgency u_p accumulates from three signals: +0.40 if $\bar{p} < 0.25$, +0.30 if the dominant query-type fraction exceeds 0.45, and +0.10 if the partition layout has not changed for more than 100 steps. A 20-step cooldown prevents partition thrashing. Urgency is zeroed if $\bar{p} > 0.50$. Every constant named here is swept in Table 6.

Neural architectures. All architectures accept input of shape (batch, W, F) and expose a common interface returning an action distribution and a scalar state-value estimate.

AttentivePPO uses a Transformer-style encoder³³: input $\rightarrow$ Dense(64, ReLU) + trainable positional encoding $\rightarrow$ multi-head self-attention (4 heads, key dimension 64, dropout 0.1) + residual + LayerNorm $\rightarrow$ FFN (128 $\rightarrow$ 64, ReLU, dropout 0.1) + residual + LayerNorm $\rightarrow$ global average pooling $\rightarrow$ actor Dense(N_a , softmax) and critic Dense(1).

MLP-PPO is a feedforward actor-critic: input $\rightarrow$ Flatten $\rightarrow$ Dense(256, ReLU) $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dropout(0.1); actor: Dense(64, ReLU) $\rightarrow$ Dense(N_a , softmax); critic: Dense(64, ReLU) $\rightarrow$ Dense(1). *MLP-matched*, used as the capacity control in *What the attention encoder actually does*, is the same topology with hidden widths solved per specialist to match *AttentivePPO*'s parameter count to within 0.06% — (128, 288) for compaction and (128, 192) for partition.

DDQN is a dueling double DQN^{34,35}: input $\rightarrow$ Flatten $\rightarrow$ Dense(256, ReLU) $\rightarrow$ BatchNorm $\rightarrow$ Dense(128, ReLU) $\rightarrow$ BatchNorm; value stream: Dense(64, ReLU) $\rightarrow$ $V(s)$; advantage stream: Dense(64, ReLU) $\rightarrow$ $A(s, a)$. The streams are aggregated using the standard mean-centred form (Eq. 6):

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right). \quad (6)$$

The Double-DQN target uses the online network to select the next action and the target network to evaluate it (Eq. 7):

$$y = r + \gamma Q_{\theta^-} \left(s', \arg \max_{a'} Q_{\theta}(s', a') \right) (1 - d), \quad (7)$$

with the target network synchronised every 50 gradient steps. For interface uniformity the model additionally exposes softmax(Q); all value updates operate on raw Q -values (this was the site of the online-adaptation defect reported in *Online adaptation*).

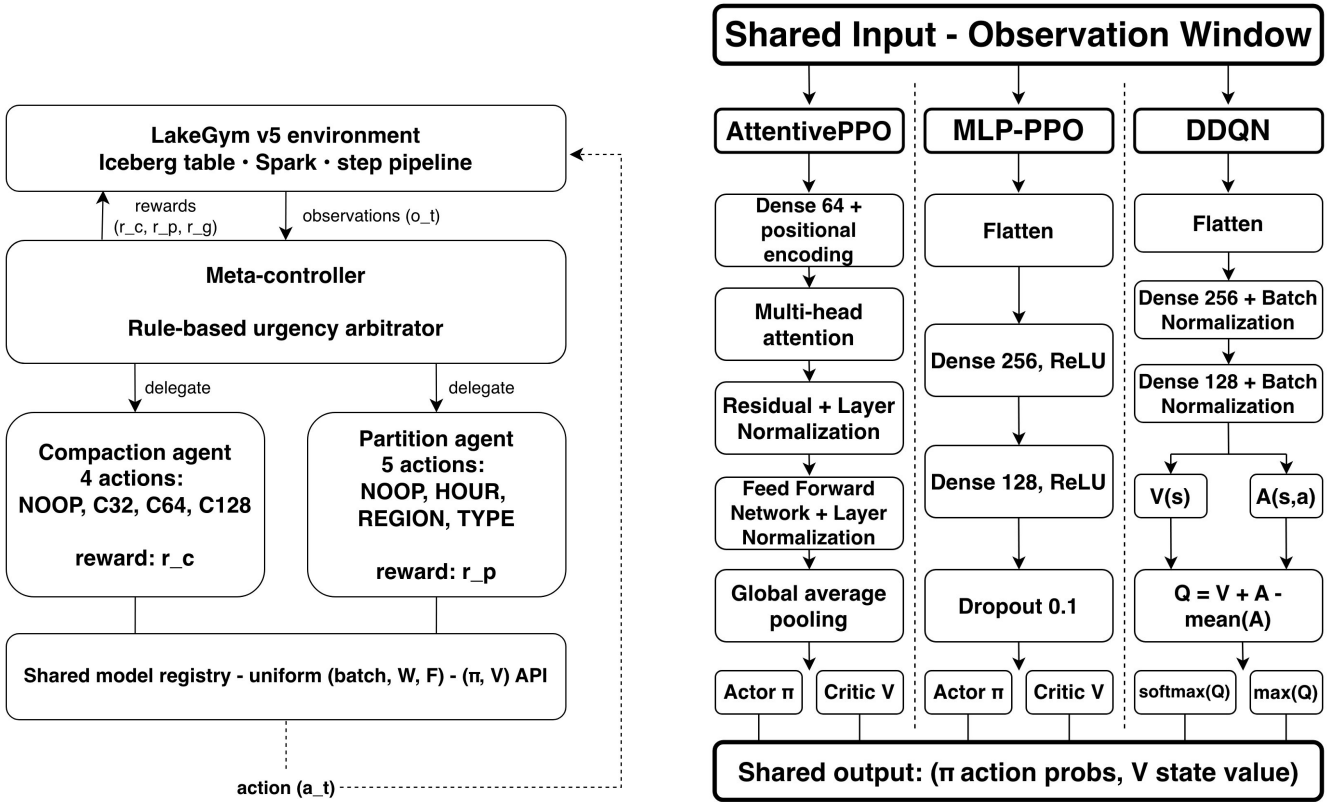


Figure 13: System architecture and neural model comparison.

Training procedure

Specialists are trained offline on logged trajectories. The compaction specialist ($W = 10$, $F = 8$, 4 actions) is trained on 14 baseline episodes comprising 17,000 transitions; the partition specialist ($W = 20$, $F = 14$, 5 actions) is trained on 6 partition-active episodes comprising 4,990 transitions. Both pools are augmented with counterfactual recovery trajectories added during this revision to break the trajectory-identity shortcut described in Results.

Three corrections to the original procedure are applied uniformly across all architectures. Staleness features are capped so they cannot encode unbounded trajectory provenance. Normalisation constants are shared between training and serving. Actions are sampled in a balanced manner during offline training, and advantages are standardised per action, so that the majority NOOP class cannot dominate the objective.

Action coverage, and why no conservative regulariser is used. The specialists are trained with standard Double DQN and offline PPO-clip, without pessimism or conservative regularisation (CQL²⁶, IQL²⁷, BCQ²⁸). Those methods exist to control value overestimation on *out-of-distribution actions*: when a fixed dataset never shows what an action does in a given region of state space, a bootstrapped value estimate for it is unconstrained, and the greedy policy is drawn towards exactly those unconstrained estimates. Balanced action sampling, which we do apply, addresses class imbalance and not this; the two are different properties, and only coverage bears on extrapolation.

We therefore report coverage directly (Table 20). Both pools contain every action of their specialist, each of them several hundred times at minimum, and — more to the point — each action appears across essentially the whole observed range of file count, the state variable that drives maintenance decisions: 10 of 10 file-count deciles for seven of the nine actions, and 9 of 10 for the other two. There is no action in either pool that the data leaves unconstrained, which is the condition under which the pessimism these methods provide would change the learned policy. The action sets are also small (4 and 5) and discrete, so there is no continuous action space in which a maximiser can drift off-support.

We state the limits of that argument plainly. Coverage is not density: the partition pool’s non-NOOP actions rest on 336–371 transitions each, drawn from only three behaviour policies, and coverage of the file-count marginal does not establish coverage of the joint state distribution. A conservative objective would be the safer default for the partition specialist on a pool that size, and we did not run one; comparing a CQL- or IQL-trained partition specialist against the present one is the natural next experiment, and we flag it as a limitation rather than claiming the choice was without cost.

Pool	Action	Count	Share	Behaviour policies	File-count deciles
Compaction (17,000)	NOOP	11,313	66.6%	12	10/10
	COMPACT_32KB	1,986	11.7%	6	10/10
	COMPACT_64KB	2,145	12.6%	8	10/10
	COMPACT_128KB	1,556	9.2%	6	9/10
Partition (4,990)	NOOP	3,570	71.5%	3	10/10
	PARTITION_HOUR	371	7.4%	3	10/10
	PARTITION_REGION	350	7.0%	3	9/10
	PARTITION_EVENT_TYPE	363	7.3%	3	10/10
	REMOVE_PARTITION	336	6.7%	2	10/10

Table 20: **Action coverage of the two offline pools.** Counts are after the action filter each specialist applies. “Behaviour policies” is the number of distinct logged policies that ever take the action, and “file-count deciles” is the number of deciles of the pool’s file-count distribution in which the action appears at least once — a coarse measure of how widely through state space the action is supported. No action is absent, and none is confined to a narrow region. Reproduced by `revision/stats/analyze_action_coverage.py`.

To make the architecture comparison in Table 3 meaningful, all three architectures use an identical configuration: the same data pools, 150 epochs for the compaction specialist and 200 for the partition specialist, batch size 64, and the same five random seeds $\{1, 2, 3, 4, 5\}$. Architecture-specific hyperparameters are listed in Table 21, and every checkpoint carries a training manifest recording the objective actually used. We note for clarity that the archive also contains an attention agent trained with advantage-weighted regression³² — the recipe used in our original submission — which is *not* the agent reported here: all results labelled **AttentivePPO** in this paper are the PPO-clip variant trained under the matched configuration above. Because the objectives differ by architecture (offline PPO-clip³⁶ for the two actor-critics, Double-DQN for the value-based model), the comparison is between architecture–objective pairings and not between encoders in isolation; the capacity-matched control in Table 11 isolates the encoder specifically.

Variant	Objective	LR	γ	Entropy	Clip / target	Params
AttentivePPO (compaction)	offline PPO-clip	3×10^{-4}	0.95	0.01	clip 0.2	84,741
AttentivePPO (partition)	offline PPO-clip	3×10^{-4}	0.99	0.10	clip 0.2	85,830
MLP-PPO (compaction)	offline PPO-clip	3×10^{-4}	0.95	0.01	clip 0.2	70,469
MLP-PPO (partition)	offline PPO-clip	3×10^{-4}	0.99	0.10	clip 0.2	121,734
DDQN (compaction)	Double DQN	1×10^{-4}	0.95	—	target 50	71,237
DDQN (partition)	Double DQN	1×10^{-4}	0.99	—	target 50	122,502

Table 21: **Hyperparameters.** All variants share: 150 epochs (compaction) / 200 epochs (partition), batch size 64, per-action advantage standardisation, balanced action sampling, and seeds $\{1, 2, 3, 4, 5\}$. Window sizes are $W = 10$ (compaction) and $W = 20$ (partition); feature counts are $F = 8$ and $F = 14$. The complete appendix is included in the archived repository.

Online adaptation. For the experiments in *Online adaptation*, agents begin from a frozen checkpoint and update weights during rollout: PPO for the two actor-critics and Double-DQN with a replay buffer for DDQN. The exploration rate decays from $\varepsilon = 0.10$ over 200 environment steps, decremented once per environment step.

Action selection differs between the frozen and adapting regimes, and the difference is not incidental. Frozen evaluation is deterministic: the specialist returns $\arg\max$ over its action distribution. During adaptation the agents must explore, so DDQN acts ε -greedily and the two actor-critics *sample* from their own softmax. A comparison between a frozen and an adapting run therefore confounds learning with action selection, which is

why the frozen control reported in *Online adaptation* was run under the adapting protocol with weight updates disabled rather than reused from Table 1.

Baseline policies

We evaluate 14 heuristic compaction policies, 3 workload-aware heuristics, and 2 ablation policies (Table 22). Heuristics include always-compact variants (C32, C64, C128), periodic compaction (every 5 or 10 steps), threshold-triggered compaction (file count > 10 or > 20), random compaction, and adaptive compaction with frequency proportional to file growth rate. Ablation policies use the **AttentivePP0** architecture restricted to the relevant action sub-space.

Policy	Description
No_Maintenance	Takes no action at any step
AlwaysCompact	Compacts every step with 64KB
AlwaysCompact_C32	Compacts every step with 32KB
AlwaysCompact_C128	Compacts every step with 128KB
Periodic5_C64	Compacts every 5 steps with 64KB
Periodic10_C64	Compacts every 10 steps with 64KB
Periodic10_C32	Compacts every 10 steps with 32KB
Periodic10_C128	Compacts every 10 steps with 128KB
Threshold10_C32	Compaction for file count > 10 (32KB)
Threshold10_C64	Compaction for file count > 10 (64KB)
Threshold10_C128	Compaction for file count > 10 (128KB)
Threshold20_C64	Compaction for file count > 20 (64KB)
Random_Compact	Uniformly samples a compaction action
AdaptiveCompaction	Proportional frequency based on file growth
WorkloadAwareThreshold	Dual-aware thresholding and partitioning
PartitionExploration	Sequential cycle of partitioning strategies
Random_Partition	Uniformly samples a partition action

Table 22: **Heuristic baseline strategies.** Fourteen rule-based compaction methods and three workload-aware benchmarks used for comparison. **WorkloadAwareThreshold** is the strongest baseline and is used as the reference in all headline comparisons.

Evaluation protocol and statistical analysis

Seeding and pairing. Each RL architecture is trained with five independent random seeds. Each resulting agent is evaluated on five episodes, giving 25 episodes per policy. Environment seeds are a deterministic function of (protocol, seed slot, episode index) and are *independent of the policy*, so every policy is evaluated on an identical set of episodes and all comparisons are paired. Heuristic policies are evaluated on the same environment seeds.

Protocols. The primary protocol is the 500-step evaluation workload (**eval500**); the 1,000-step standard workload (**std1000**) is reported as a concordance check (Spearman $\rho = 0.964$). Held-out protocols **drift500** and **mixed500** are described above.

The sub-studies in this paper do *not* all run at the same budget: the sensitivity sweeps and the oracle experiment are far more expensive per configuration than a single policy evaluation, and were scoped accordingly. Table 23 states exactly which protocol produced every reported number, because several quantities that look comparable are not. In particular, DDQN appears at three different values in this paper — 0.2105, 0.2033 and 0.2306 — and all three are the same agent under the rule-based meta-controller, measured under three different protocols. Each results subsection names its protocol in its opening sentence, and every table caption repeats it.

Statistical methods. Uncertainty on means is reported as percentile bootstrap 95% confidence intervals with 10,000 resamples³⁷, computed over independent episodes and never over autocorrelated within-episode steps.

We report *two* intervals, because the evaluation design is hierarchical: 25 episodes are nested within 5 training seeds, and episodes sharing a seed share a policy. An ordinary bootstrap over the pooled 25 episodes treats seed-level and episode-level variation as exchangeable, and therefore estimates the precision of *this* agent’s mean rather than the performance of a newly trained one. We therefore also report a *cluster bootstrap*³⁸ that resamples the five training seeds with replacement, carrying each drawn seed’s episodes with it. This is the appropriate interval for the quantity a practitioner cares about, and it is wider — for DDQN, [0.2026, 0.2185] against [0.2050, 0.2155]. It is

Reported in	Protocol	Steps	Seeds	Episodes/seed	Action selection
Tables 1, 2, 3	eval500	500	5	5	greedy arg max
Table 4 (ablations)	eval500	500	5 / 3 [‡]	5	greedy arg max
Table 6 (meta-controller sweep)	eval500	500	3	1	greedy arg max
Table 7 (pruning scale)	eval500	500	3	1	greedy arg max
Table 8 (oracle)	std1000	1,000	1 [§]	1	one-step rollback oracle
Table 9 (held-out)	drift500 / mixed500	500	5 / 3 [‡]	1	greedy arg max
Table 10 (adaptation)	eval500 plan	500	5	5 consecutive	sampled / ϵ -greedy
Table 10 (control arm A)	eval500 plan	500	5	5 consecutive	sampled / ϵ -greedy
Table 10 (control arm B)	eval500 plan	500	5	5 consecutive	greedy arg max
Table 5 (DDQN ablation)	eval500	500	5 / 3 [‡]	5 / 1 [‡]	greedy arg max
Table 11 (capacity control)	eval500	500	5	5	greedy arg max

Table 23: **Evaluation protocol for every reported result.** Protocol names map one-to-one onto the workload plans in the archived repository: `std1000` $\rightarrow$ `workload_plan_v5.json`, `eval500` $\rightarrow$ `workload_plan_v5_eval.json`, `drift500` $\rightarrow$ `workload_plan_v5_drift.json`, `mixed500` $\rightarrow$ `workload_plan_v5_mixed.json`. Environment seeds are a deterministic function of (protocol, seed slot, episode) and are independent of the policy, so policies evaluated under the same protocol are compared on identical episodes. Different protocols draw from *different* environment-seed blocks, so numbers from different rows of this table are not paired and small differences between them carry no meaning. [‡]Heuristics and `PartitionOnly` were run on 3 seeds where their across-seed variability is negligible (s.d. 0.0002–0.002) or the runs are prohibitively slow — a `PartitionOnly` episode costs 36 minutes of Spark execution against 7 for an ordinary policy, because the table it leaves behind holds over a thousand files. [§]The oracle requires three trial branches and a rollback per step; one 1,000-step episode costs two hours of Spark execution, so it was run on one seed.

quoted in the Abstract and reported alongside the episode-level interval in Table 1. With five clusters the interval is necessarily coarse; that coarseness is an honest reflection of how many independent training runs were performed, not a defect of the estimator. Across-seed standard deviation is reported in addition, because it exposes the spread the interval summarises.

Paired comparisons use both the Wilcoxon signed-rank test and a paired permutation test (exact for $n \leq 20$, Monte Carlo with 20,000 sign-flips above that, giving a smallest attainable raw p of 5×10^{-5}); effect sizes are Cliff’s δ^{39} with thresholds 0.147 (small), 0.33 (medium) and 0.474 (large). Pairing is resolved per comparison, on the environment seeds the two policies share, so a policy evaluated on fewer seeds does not silently truncate unrelated comparisons. Multiplicity is controlled with Holm–Bonferroni correction⁴⁰ within each comparison family. As a secondary check at the level the cluster bootstrap uses, the headline comparison was repeated on the five per-seed means: all five DDQN seeds exceed the best heuristic ($\delta = +1.00$), which is the most extreme outcome the design can produce, and the exact permutation test on $n = 5$ therefore returns its floor of $p = 0.0625$. We report that floor rather than presenting the seed-level test as evidence of no effect. Where a design cannot reach $p < 0.05$ — the sensitivity sweep ($n = 3$, floor $p = 0.25$) and the generalization study ($n = 5$, floor $p = 0.0625$) — we report intervals and effect sizes and state that the study is powered for estimation rather than hypothesis testing.

Data integrity. A Spark driver crash does not raise inside the simulator: metric collection returns zero and the query returns a latency of -1 , which the reward formula maps to exactly 0.0, so a dead run can be written out with a successful status. We therefore audit every run before analysis with a validator that rejects any episode with non-positive mean latency, zero mean file count, absurd latency ($> 60,000$ ms), or identical latency across all episodes of a seed. All 216 evaluation runs reported here pass; runs that failed were deleted and re-executed under per-seed process isolation. The validator is included in the archived repository.

Cost model. $C = C_q \sum_t \ell_t + C_c N_c + C_s \sum_t \sigma_t$, where $C_q = \$0.00001/\text{ms}$, $C_c = \$0.005$ per compaction operation, $C_s = \$0.000001/\text{KB-step}$, ℓ_t is per-step latency, N_c is total compaction operations, and σ_t is table size in KB at step t . Table 15 adds inference at $\$0.096/\text{h}$ CPU and offline training at $\$0.526/\text{h}$ GPU amortised over 100 deployment episodes. We note that a flat per-operation compaction charge under-charges aggressive strategies relative to production, where a compaction is a distributed rewrite whose cost grows with bytes touched; we therefore also evaluate a size-proportional variant calibrated to the same mean, which leaves the ordering essentially unchanged at this simulation’s scale.

References

1. Blue, R., Weeks, D., Reid, J. & Steinbach, C. Iceberg: A fast and expressive table format for large, slow tables. In *Proc. ACM SIGMOD* 3434–3441 (ACM, 2020).
2. Armbrust, M. *et al.* Delta Lake: High-performance ACID table storage over cloud object stores. *Proc. VLDB Endow.* **13**, 3411–3424 (2020).
3. Chandar, V., Rao, B. S. & Agarwal, N. Apache Hudi: The data lake platform. *arXiv preprint arXiv:2301.09517* (2023).
4. Kraska, T. *et al.* The case for learned index structures. In *Proc. ACM SIGMOD* 489–504 (ACM, 2018).
5. Agrawal, S. *et al.* Database tuning advisor for Microsoft SQL Server 2005. In *Proc. VLDB* 1110–1121 (2004).
6. Marcus, R. & Papaemmanouil, O. Re-Join: How the machine learned to join. In *Workshop on ML for Progressively Learned Databases (NIPS DBLearn)* (2018).
7. Trummer, I. *et al.* SkinnerDB: Regret-bounded query evaluation via reinforcement learning. In *Proc. ACM SIGMOD* 1153–1170 (ACM, 2019).
8. Van Aken, D., Pavlo, A., Gordon, G. J. & Zhang, B. Automatic database management system tuning through large-scale machine learning. In *Proc. ACM SIGMOD* 1009–1024 (ACM, 2017).
9. Zhang, J. *et al.* An end-to-end automatic cloud database tuning system using deep reinforcement learning. In *Proc. ACM SIGMOD* 415–432 (ACM, 2019).
10. de Haan, P., Jayaraman, D. & Levine, S. Causal confusion in imitation learning. In *Adv. Neural Inf. Process. Syst.* **32**, 11698–11709 (2019).
11. Wen, C., Lin, J., Darrell, T., Jayaraman, D. & Gao, Y. Fighting copycat agents in behavioral cloning from observation histories. In *Adv. Neural Inf. Process. Syst.* **33**, 2564–2575 (2020).
12. Yang, Z. *et al.* Qd-tree: Learning Data Layouts for Big Data Analytics. In *Proc. ACM SIGMOD* 170–184 (ACM, 2020).
13. Kraska, T. *et al.* SageDB: A Learned Database System. In *Proc. CIDR* (2019).
14. Li, G. *et al.* Q Tune: A Query-Aware Database Tuning System with Deep Reinforcement Learning. *Proc. VLDB Endow.* **12**, 2118–2130 (2019).
15. Kanellis, K. *et al.* LlamaTune: Sample-Efficient DBMS Configuration Tuning. *Proc. VLDB Endow.* **15**, 2726–2738 (2022).
16. Cai, B. *et al.* HUNTER: An Online Cloud Database Hybrid Tuning System for Personalized Requirements. In *Proc. ACM SIGMOD* 625–638 (ACM, 2022).
17. Yang, Z. *et al.* HWTune: A Hardware-Aware Database Tuning System with Deep Reinforcement Learning. *IEEE Trans. Knowl. Data Eng.* **36**, 2284–2297 (2023).
18. Feng, Y. *et al.* CTuner: Automatic NoSQL Database Tuning with Causal Reinforcement Learning. In *Proc. ACM SIGKDD* 1–12 (ACM, 2024).
19. Liu, Y. *et al.* MLETune: Streamlining Database Knob Tuning via Multi-LLMs Experts Guided Deep Reinforcement Learning. *IEEE Access* **12**, 1–15 (2024).
20. Wang, Y. *et al.* A highly efficient distributed database knobs tuning method based on multi-agent deep reinforcement learning. *J. Supercomput.* (2025).
21. Pavlo, A. *et al.* Self-Driving Database Management Systems. In *Proc. CIDR* (2017).
22. Marcus, R. *et al.* Neo: A Learned Query Optimizer. *Proc. VLDB Endow.* **12**, 1705–1718 (2019).
23. Camacho-Rodríguez, J. *et al.* LST-Bench: Benchmarking Log-Structured Tables in the Cloud. In *Proc. ACM SIGMOD* 1–14 (ACM, 2024).

24. Armbrust, M. *et al.* Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *Proc. CIDR* (2021).
25. Dayan, N., Athanassoulis, M. & Idreos, S. Dostoevsky: Better space-time trade-offs for LSM-tree based key-value stores via adaptive removal of superfluous merging. In *Proc. ACM SIGMOD* 505–520 (ACM, 2018).
26. Kumar, A., Zhou, A., Tucker, G. & Levine, S. Conservative Q-learning for offline reinforcement learning. In *Adv. Neural Inf. Process. Syst.* **33**, 1179–1191 (2020).
27. Kostrikov, I., Nair, A. & Levine, S. Offline reinforcement learning with implicit Q-learning. In *Proc. Int. Conf. Learn. Represent.* (2022).
28. Fujimoto, S., Meger, D. & Precup, D. Off-policy deep reinforcement learning without exploration. In *Proc. Int. Conf. Mach. Learn.* **97**, 2052–2062 (2019).
29. Vezhnevets, A. S. *et al.* FeUdal Networks for Hierarchical Reinforcement Learning. In *Proc. ICML* 3540–3549 (2017).
30. Bacon, P.-L., Harb, J. & Precup, D. The option-critic architecture. In *Proc. AAAI* 1726–1734 (2017).
31. Zhang, K., Yang, Z. & Başar, T. Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms. *J. Mach. Learn. Res.* **22**, 1–85 (2021).
32. Peng, X. B., Kumar, A., Zhang, G. & Levine, S. Advantage-weighted regression: simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177* (2019).
33. Vaswani, A. *et al.* Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)* vol. 30 (2017).
34. Wang, Z. *et al.* Dueling network architectures for deep reinforcement learning. In *Proc. ICML* 1995–2003 (2016).
35. van Hasselt, H., Guez, A. & Silver, D. Deep reinforcement learning with double Q-learning. In *Proc. AAAI* 2094–2100 (2016).
36. Schulman, J. *et al.* Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
37. Efron, B. & Tibshirani, R. J. *An Introduction to the Bootstrap* (Chapman & Hall, 1993).
38. Field, C. A. & Welsh, A. H. Bootstrapping clustered data. *J. R. Stat. Soc. B* **69**, 369–390 (2007).
39. Cliff, N. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychol. Bull.* **114**, 494–509 (1993).
40. Holm, S. A simple sequentially rejective multiple test procedure. *Scand. J. Stat.* **6**, 65–70 (1979).

Data availability

The datasets generated and analysed during the current study, including all experimental metrics collected across workload profiles, every per-run manifest, and the model weights for all seeds, are archived at Zenodo under DOI 10.5281/zenodo.22868593 (release **v1.1-revision2**; the version-independent DOI 10.5281/zenodo.21860272 always resolves to the most recent version) and are additionally available in the LakePilot repository at <https://github.com/OmarEssameldinMousa/LakePilot-Research-prototype>. No pre-existing external datasets were used in this study; all data was synthetically generated by the LAKEGYM v5 environment as described in the Methods section.

Code availability

The LAKEGYM v5 simulation environment, the meta-controller implementation, the model registry, all training and evaluation scripts, the statistical analysis pipeline, and the data-integrity validator are archived at Zenodo under DOI 10.5281/zenodo.22868593 (release v1.1-revision2; the version-independent DOI 10.5281/zenodo.21860272 always resolves to the most recent version) and are available under the MIT licence at <https://github.com/0marEssameldinMousa/LakePilot-Research-prototype>. The deposit pins the exact software environment used for every reported result (Python 3.10.12, Apache Spark 3.5.7, PySpark 3.5.7, TensorFlow 2.20.0, NumPy 2.2.6, pandas 2.3.3) and includes end-to-end reproduction instructions. Every run carries a `manifest.json` recording its random seed, configuration, git commit, hardware and wall-clock time.

Acknowledgements

The authors developed the LAKEGYM v5 simulation environment utilising the open-source Apache Iceberg and Apache Spark frameworks. All figures were generated using Matplotlib (v3.8, <https://matplotlib.org>).

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Author contributions statement

O.E. conceived the idea, implemented the LAKEGYM v5 environment, developed the lakehouse solution, and designed the whole experiments. O.K. implemented and trained the AI models. O.S. assisted in baseline analysis and overall project execution. A.Z.B. supervised the research. All authors reviewed and approved the final manuscript.

Additional information

Competing interests. The authors declare no competing interests.